

Lab1_2

June 11, 2026

1 Lab 01 : Data preprocessing and visualisation

1.1 Task 1 : Understanding Data

```
[1]: import pandas as pd
import numpy as np
```

```
[2]: city= pd.read_csv('./data/city_day.csv')
city.head()
```

```
[2]:
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	S02	\
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	

	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	133.36	0.00	0.02	0.00	NaN	NaN
1	34.06	3.68	5.50	3.77	NaN	NaN
2	30.70	6.80	16.40	2.25	NaN	NaN
3	36.08	4.43	10.14	1.00	NaN	NaN
4	39.31	7.01	18.89	2.78	NaN	NaN

```
[3]: crop=pd.read_csv('./data/crop_production.csv')
crop.head()
```

```
[3]:
```

	State_Name	District_Name	Crop_Year	Season	\
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	

	Crop	Area	Production
0	Arecanut	1254.0	2000.0
1	Other Kharif pulses	2.0	1.0
2	Rice	102.0	321.0

3	Banana	176.0	641.0
4	Cashewnut	720.0	165.0

1.1.1 City Dataset

Structure and Shape

```
[4]: city.shape
```

```
[4]: (29531, 16)
```

```
[5]: city.dtypes
```

```
[5]: City          object
Date            object
PM2.5          float64
PM10           float64
NO             float64
NO2            float64
NOx            float64
NH3            float64
CO             float64
SO2            float64
O3             float64
Benzene         float64
Toluene         float64
Xylene          float64
AQI             float64
AQI_Bucket      object
dtype: object
```

```
[6]: city.columns
```

```
[6]: Index(['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
        'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket'],
        dtype='object')
```

Summary Statistics

```
[7]: city.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  -
0   City            29531 non-null  object
1   Date            29531 non-null  object
2   PM2.5           24933 non-null  float64
3   PM10            18391 non-null  float64
```

```

4   NO          25949 non-null float64
5   NO2         25946 non-null float64
6   NOx         25346 non-null float64
7   NH3         19203 non-null float64
8   CO          27472 non-null float64
9   SO2         25677 non-null float64
10  O3          25509 non-null float64
11  Benzene     23908 non-null float64
12  Toluene     21490 non-null float64
13  Xylene      11422 non-null float64
14  AQI         24850 non-null float64
15  AQI_Bucket  24850 non-null object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB

```

```
[8]: city.describe()
```

```

[8]:
      count      PM2.5      PM10      NO      NO2      NOx  \
count  24933.000000  18391.000000  25949.000000  25946.000000  25346.000000
mean    67.450578    118.127103    17.574730    28.560659    32.309123
std     64.661449     90.605110    22.785846    24.474746    31.646011
min      0.040000     0.010000     0.020000     0.010000     0.000000
25%     28.820000     56.255000     5.630000    11.750000    12.820000
50%     48.570000     95.680000     9.890000    21.690000    23.520000
75%     80.590000    149.745000    19.950000    37.620000    40.127500
max     949.990000   1000.000000    390.680000   362.210000   467.630000

      count      NH3      CO      SO2      O3      Benzene  \
count  19203.000000  27472.000000  25677.000000  25509.000000  23908.000000
mean    23.483476     2.248598    14.531977    34.491430     3.280840
std     25.684275     6.962884    18.133775    21.694928    15.811136
min      0.010000     0.000000     0.010000     0.010000     0.000000
25%      8.580000     0.510000     5.670000    18.860000     0.120000
50%     15.850000     0.890000     9.160000    30.840000     1.070000
75%     30.020000     1.450000    15.220000    45.570000     3.080000
max     352.890000    175.810000    193.860000   257.730000   455.030000

      count      Toluene      Xylene      AQI
count  21490.000000  11422.000000  24850.000000
mean     8.700972     3.070128    166.463581
std     19.969164     6.323247    140.696585
min      0.000000     0.000000    13.000000
25%      0.600000     0.140000    81.000000
50%      2.970000     0.980000   118.000000
75%      9.150000     3.350000   208.000000
max     454.850000    170.370000   2049.000000

```

```
[9]: city.describe(include='object')
```

```
[9]:
```

	City	Date	AQI_Bucket
count	29531	29531	24850
unique	26	2009	6
top	Ahmedabad	2020-06-26	Moderate
freq	2009	26	8829

Missing Values

```
[10]: city.isna().sum()
```

```
[10]: City          0
      Date          0
      PM2.5        4598
      PM10        11140
      NO           3582
      NO2          3585
      NOx          4185
      NH3         10328
      CO           2059
      SO2          3854
      O3           4022
      Benzene       5623
      Toluene       8041
      Xylene       18109
      AQI           4681
      AQI_Bucket    4681
      dtype: int64
```

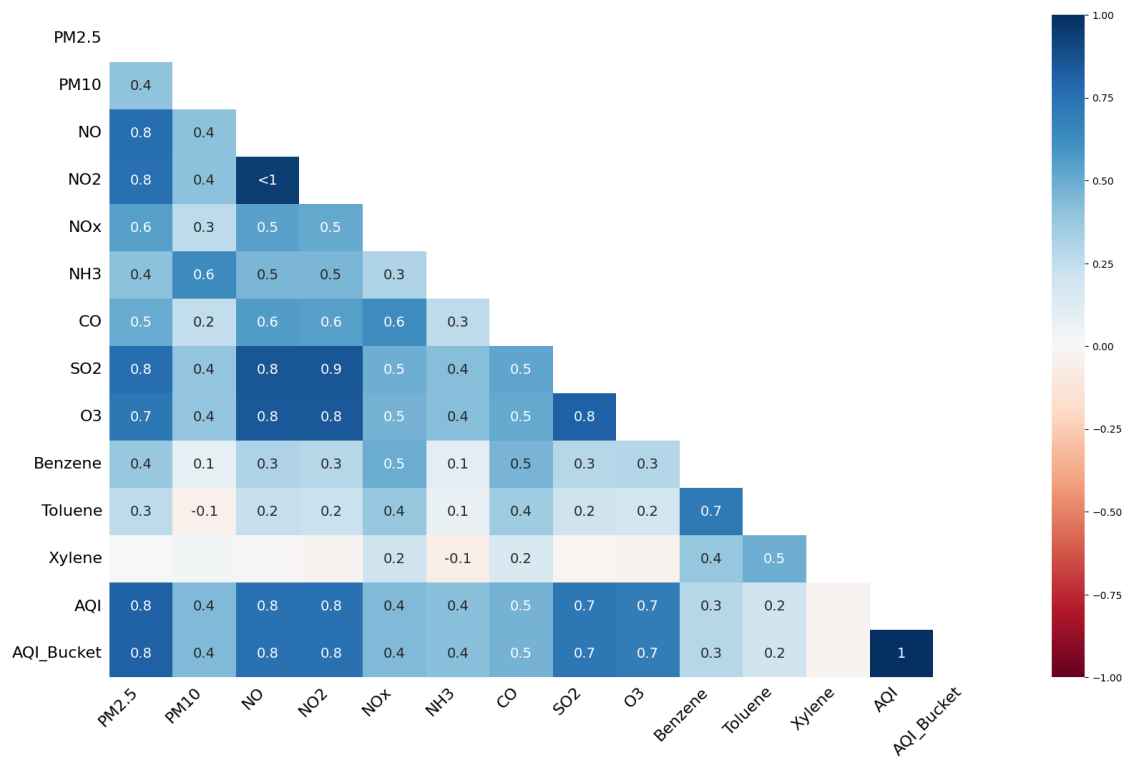
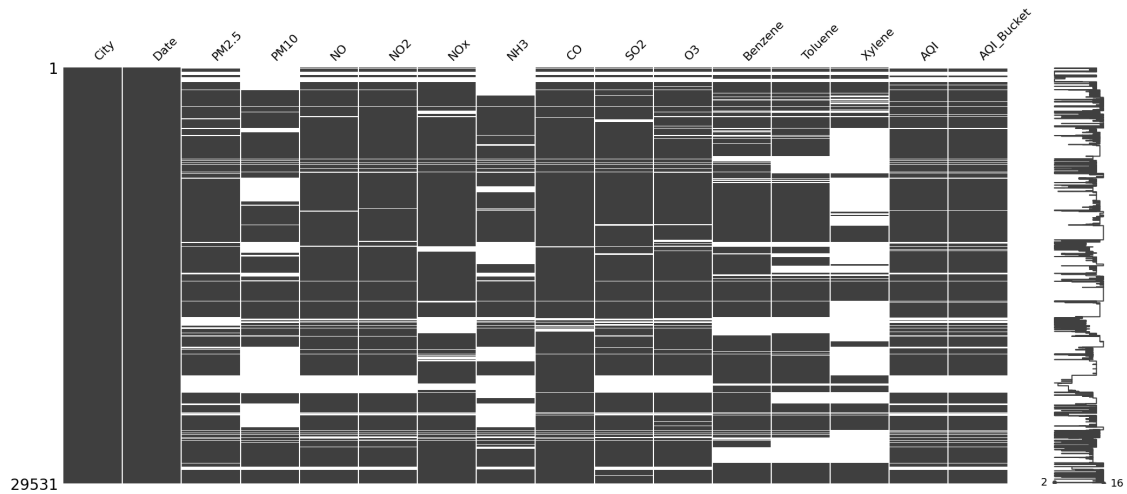
```
[11]: ((city.isna().sum())/len(city))* 100
```

```
[11]: City          0.000000
      Date          0.000000
      PM2.5        15.570079
      PM10        37.723071
      NO           12.129626
      NO2          12.139785
      NOx          14.171549
      NH3          34.973418
      CO           6.972334
      SO2          13.050692
      O3           13.619586
      Benzene       19.041008
      Toluene       27.229014
      Xylene       61.322001
      AQI           15.851139
      AQI_Bucket    15.851139
      dtype: float64
```

```
[12]: import missingno as msno
```

```
msno.matrix(city)
msno.heatmap(city)
```

```
[12]: <Axes: >
```



Duplicate Values

```
[13]: city.duplicated().sum()
```

```
[13]: np.int64(0)
```

Understanding data

```
[14]: city.nunique()
```

```
[14]: City          26
      Date          2009
      PM2.5        11716
      PM10         12571
      NO           5776
      NO2          7404
      NOx          8156
      NH3          5922
      CO           1779
      SO2          4761
      O3           7699
      Benzene      1873
      Toluene      3608
      Xylene       1561
      AQI          829
      AQI_Bucket    6
      dtype: int64
```

```
[15]: city['AQI_Bucket'].unique()
```

```
[15]: array([nan, 'Poor', 'Very Poor', 'Severe', 'Moderate', 'Satisfactory',
          'Good'], dtype=object)
```

```
[16]: city.select_dtypes(include='number').nunique().sort_values()
```

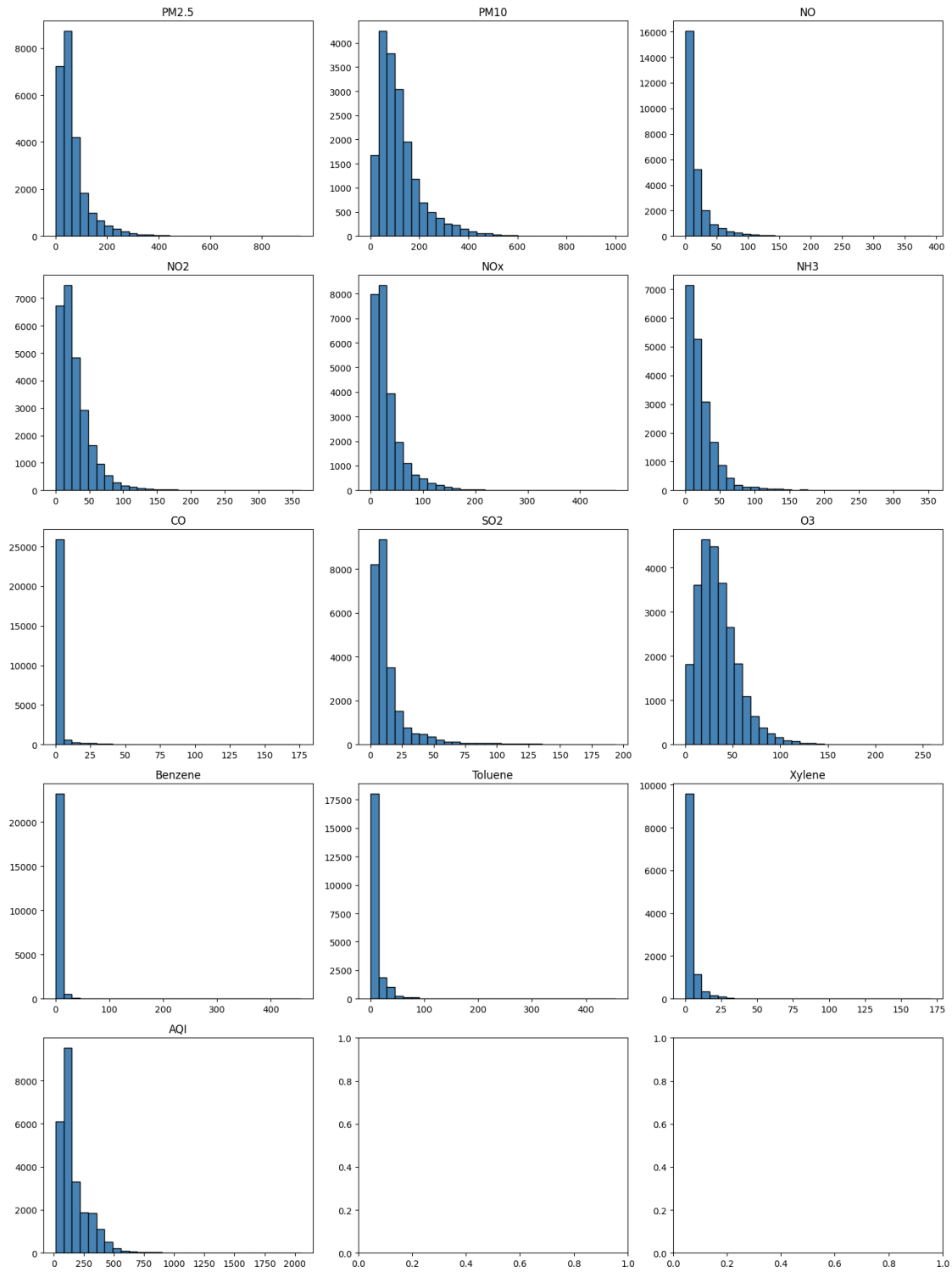
```
[16]: AQI          829
      Xylene      1561
      CO          1779
      Benzene     1873
      Toluene     3608
      SO2         4761
      NO          5776
      NH3         5922
      NO2         7404
      O3          7699
      NOx         8156
      PM2.5       11716
      PM10        12571
      dtype: int64
```

```
[17]: import matplotlib.pyplot as plt

num_cols = city.select_dtypes(include='number').columns.tolist()
fig, axes = plt.subplots(5, 3, figsize=(15, 20))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    axes[i].hist(city[col].dropna(), bins=30, color='steelblue',
    ↪edgecolor='black')
    axes[i].set_title(col)

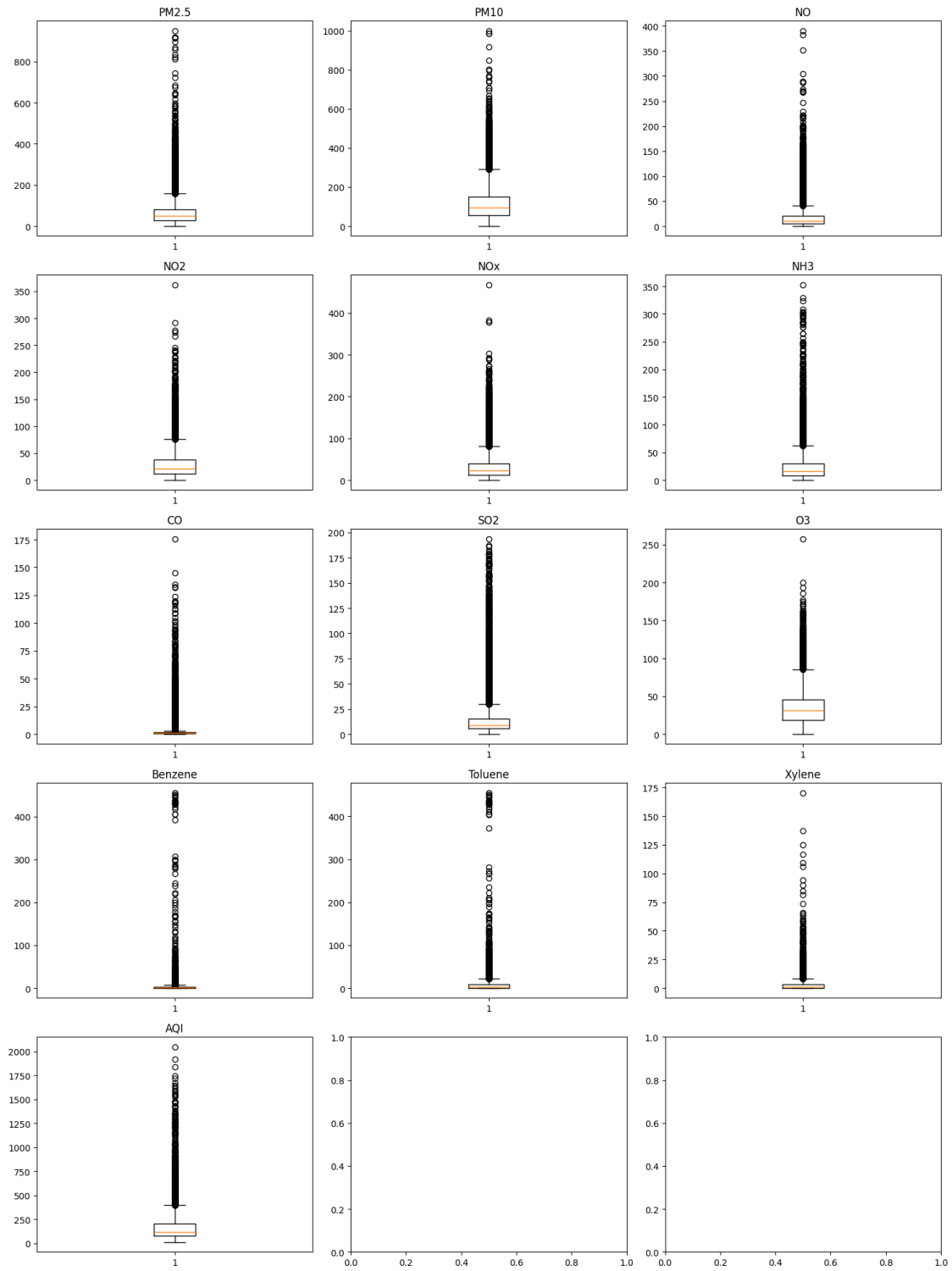
plt.tight_layout()
plt.show()
```



```
[18]: fig, axes = plt.subplots(5, 3, figsize=(15, 20))
      axes = axes.flatten()
```



```
for i, col in enumerate(num_cols):  
    axes[i].boxplot(city[col].dropna())  
    axes[i].set_title(col)  
  
plt.tight_layout()  
plt.show()
```



Outlier Detection

```
[19]: # IQR method
Q1 = city.select_dtypes(include='number').quantile(0.25)
Q3 = city.select_dtypes(include='number').quantile(0.75)
IQR = Q3 - Q1

outliers = ((city.select_dtypes(include='number') < (Q1 - 1.5 * IQR)) |
            (city.select_dtypes(include='number') > (Q3 + 1.5 * IQR))).sum()
print(outliers)
```

```
PM2.5      1982
PM10       1057
NO         2459
NO2        1188
NOx        1868
NH3        1015
CO         2475
SO2        2578
O3         713
Benzene    1668
Toluene    2427
Xylene     1119
AQI        1358
dtype: int64
```

Data Profile

Attribute	Detail
Shape	29,531 rows × 16 columns
Numerical columns	13 (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene, Xylene, AQI)
Categorical columns	3 (City, Date, AQI_Bucket)
Cities covered	26
Date range	2,009 unique dates
Duplicate rows	0
Target column	AQI (with 6 categories: Good, Satisfactory, Moderate, Poor, Very Poor, Severe)

Missing Values Summary

Column	Missing Count	Missing %
Xylene	18,109	61.3%
PM10	11,140	37.7%
NH3	10,328	34.9%
Toluene	8,041	27.2%
Benzene	5,623	19.0%
AQI / AQI_Bucket	4,681	15.8%

Column	Missing Count	Missing %
PM2.5	4,598	15.5%

Outliers (IQR Method)

Column	Outlier Count
CO	2,475
SO2	2,578
NO	2,459
Toluene	2,427
PM2.5	1,982
NOx	1,868
Benzene	1,668
AQI	1,358

Distribution Shape (from Histograms) All 13 numerical columns show strong right skew — the majority of readings are low, with a long tail of extreme pollution events. AQI max reaches 2049, while the median is only 118.

Concern Xylene is missing in 61.3% of rows and may not be usable as a feature. More critically, PM2.5 and PM10 — the two most health-relevant pollutants — are missing in 15–38% of rows respectively. Every pollutant column contains significant outliers, with AQI reaching a maximum of 2,049 against a median of 118, suggesting extreme pollution events that could distort any model trained on this data without treatment.

1.1.2 Crop Dataset

Structure and Shape

```
[20]: crop.shape
```

```
[20]: (246091, 7)
```

```
[21]: crop.dtypes
```

```
[21]: State_Name      object
      District_Name  object
      Crop_Year       int64
      Season          object
      Crop            object
      Area            float64
      Production      float64
      dtype: object
```

```
[22]: crop.columns
```

```
[22]: Index(['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area',
          'Production'],
          dtype='object')
```

Summary Statistics

```
[23]: crop.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   State_Name      246091 non-null object
1   District_Name   246091 non-null object
2   Crop_Year       246091 non-null int64
3   Season          246091 non-null object
4   Crop            246091 non-null object
5   Area            246091 non-null float64
6   Production      242361 non-null float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
```

```
[24]: crop.describe()
```

```
[24]:
```

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

```
[25]: crop.describe(include='object')
```

```
[25]:
```

	State_Name	District_Name	Season	Crop
count	246091	246091	246091	246091
unique	33	646	6	124
top	Uttar Pradesh	BIJAPUR	Kharif	Rice
freq	33306	945	95951	15104

Missing Values

```
[26]: crop.isna().sum()
```

```
[26]: State_Name      0
      District_Name   0
      Crop_Year       0
```

```

Season          0
Crop            0
Area           0
Production      3730
dtype: int64

```

```
[27]: ((crop.isna().sum())/len(crop))* 100
```

```

[27]: State_Name      0.000000
      District_Name  0.000000
      Crop_Year      0.000000
      Season         0.000000
      Crop           0.000000
      Area           0.000000
      Production     1.515699
      dtype: float64

```

Duplicate Values

```
[28]: crop.duplicated().sum()
```

```
[28]: np.int64(0)
```

Understanding data

```
[29]: crop.nunique()
```

```

[29]: State_Name      33
      District_Name  646
      Crop_Year      19
      Season         6
      Crop           124
      Area           38442
      Production     51627
      dtype: int64

```

```
[30]: crop.select_dtypes(include='number').nunique().sort_values()
```

```

[30]: Crop_Year      19
      Area           38442
      Production     51627
      dtype: int64

```

```
[31]: crop['Season'].unique()
```

```

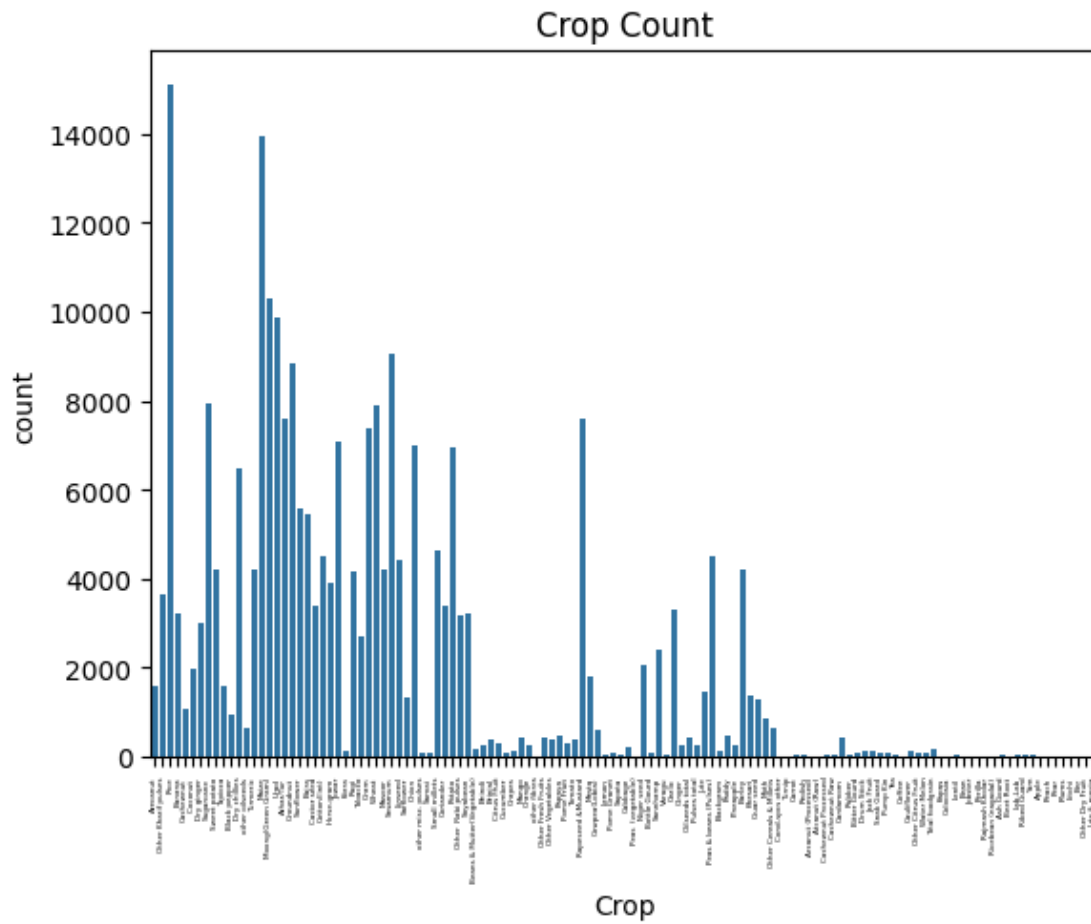
[31]: array(['Kharif      ', 'Whole Year ', 'Autumn      ', 'Rabi          ',
            'Summer      ', 'Winter      '], dtype=object)

```

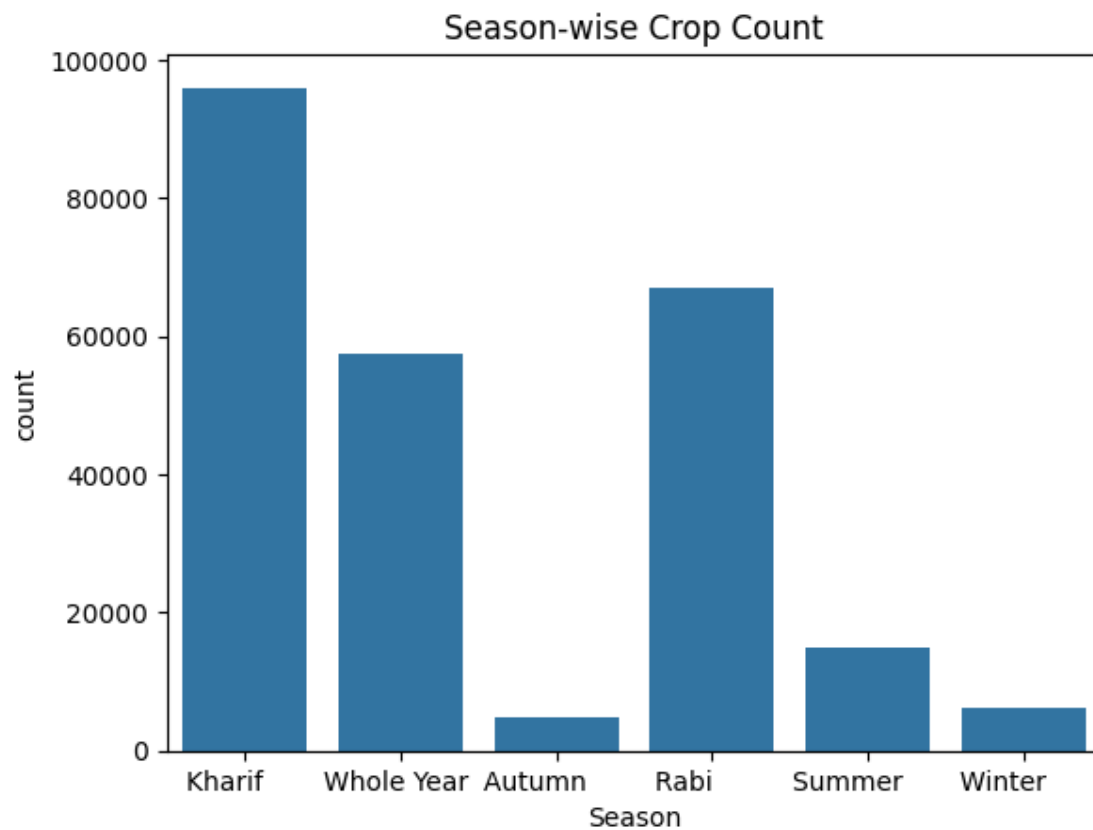
```
[32]: crop['Crop'].unique()
```

```
[32]: array(['Arecanut', 'Other Kharif pulses', 'Rice', 'Banana', 'Cashewnut',
'Coconut ', 'Dry ginger', 'Sugarcane', 'Sweet potato', 'Tapioca',
'Black pepper', 'Dry chillies', 'other oilseeds', 'Turmeric',
'Maize', 'Moong(Green Gram)', 'Urad', 'Arhar/Tur', 'Groundnut',
'Sunflower', 'Bajra', 'Castor seed', 'Cotton(lint)', 'Horse-gram',
'Jowar', 'Korra', 'Ragi', 'Tobacco', 'Gram', 'Wheat', 'Masoor',
'Sesamum', 'Linseed', 'Safflower', 'Onion', 'other misc. pulses',
'Samai', 'Small millets', 'Coriander', 'Potato',
'Other Rabi pulses', 'Soyabean', 'Beans & Mutter(Vegetable)',
'Bhindi', 'Brinjal', 'Citrus Fruit', 'Cucumber', 'Grapes', 'Mango',
'Orange', 'other fibres', 'Other Fresh Fruits', 'Other Vegetables',
'Papaya', 'Pome Fruit', 'Tomato', 'Rapeseed &Mustard', 'Mesta',
'Cowpea(Lobia)', 'Lemon', 'Pome Granet', 'Sapota', 'Cabbage',
'Peas (vegetable)', 'Niger seed', 'Bottle Gourd', 'Sannhamp',
'Varagu', 'Garlic', 'Ginger', 'Oilseeds total', 'Pulses total',
'Jute', 'Peas & beans (Pulses)', 'Blackgram', 'Paddy', 'Pineapple',
'Barley', 'Khesari', 'Guar seed', 'Moth',
'Other Cereals & Millets', 'Cond-spcs other', 'Turnip', 'Carrot',
'Redish', 'Arcanut (Processed)', 'Atcanut (Raw)',
'Cashewnut Processed', 'Cashewnut Raw', 'Cardamom', 'Rubber',
'Bitter Gourd', 'Drum Stick', 'Jack Fruit', 'Snak Guard',
'Pump Kin', 'Tea', 'Coffee', 'Cauliflower', 'Other Citrus Fruit',
'Water Melon', 'Total foodgrain', 'Kapas', 'Colocosia', 'Lentil',
'Bean', 'Jobster', 'Perilla', 'Rajmash Kholar',
'Ricebean (nagadal)', 'Ash Gourd', 'Beet Root', 'Lab-Lab',
'Ribed Guard', 'Yam', 'Apple', 'Peach', 'Pear', 'Plums', 'Litchi',
'Ber', 'Other Dry Fruit', 'Jute & mesta'], dtype=object)
```

```
[33]: import seaborn as sns
sns.countplot(x='Crop', data = crop)
plt.title("Crop Count")
plt.xticks(rotation=90, fontsize=3)
plt.show()
```

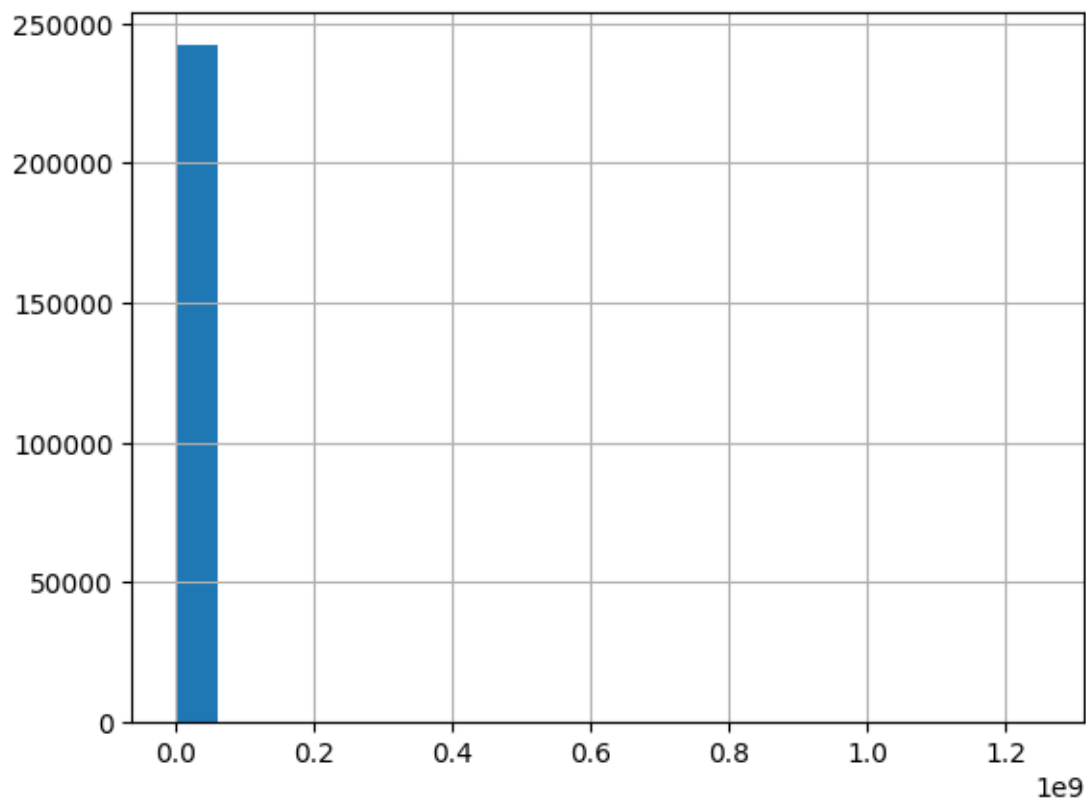


```
[34]: import seaborn as sns
sns.countplot(x='Season', data = crop)
plt.title("Season-wise Crop Count")
plt.show()
```

```
[35]: crop['Production'].hist(bins=20)
```

```
[35]: <Axes: >
```



Issues with Production distribution must be handled later through normalisation.

Outlier Detection

```
[36]: # IQR method
Q1 = crop.select_dtypes(include='number').quantile(0.25)
Q3 = crop.select_dtypes(include='number').quantile(0.75)
IQR = Q3 - Q1

outliers = ((crop.select_dtypes(include='number') < (Q1 - 1.5 * IQR)) |
            (crop.select_dtypes(include='number') > (Q3 + 1.5 * IQR))).sum()
print(outliers)
```

```
Crop_Year      0
Area          40705
Production     42390
dtype: int64
```

Data Profile

Attribute	Detail
Shape	246,091 rows × 7 columns

Attribute	Detail
Numerical columns	3 (Crop_Year, Area, Production)
Categorical columns	4 (State_Name, District_Name, Season, Crop)
States covered	33
Districts covered	646
Year range	1997 – 2015 (19 unique years)
Unique crops	124
Unique seasons	6
Duplicate rows	0

Missing Values Summary

Column	Missing Count	Missing %
Production	3,730	1.5%
All others	0	0%

1.1.3 Outliers (IQR Method)

Column	Outlier Count
Area	40,705
Production	42,390

Key Observations from Distributions

- **Season:** Kharif dominates (~96,000 records), followed by Rabi (~67,000). Autumn and Winter are sparsely represented.
- **Crop:** Rice is the most recorded crop (15,104 records). Many crops appear very rarely — potential class imbalance issue.
- **Area & Production:** Extreme right skew with massive outlier counts. Mean production (582,503) is astronomically higher than median (729), indicating a few large-scale crops pulling the average up heavily.

Concern Area and Production have over 40,000 outliers each — nearly 16% of all rows. This is not noise; it reflects genuine variation between small subsistence farms and large commercial operations. However, the gap between mean (582,503) and median (729) in Production is extreme and will severely distort any ML model if left untreated. A log transformation will likely be necessary before modelling. Additionally, Season values contain trailing spaces (e.g. ‘Kharif’) which must be cleaned before the dataset can be merged with the city file.

1.2 Task 2 : Handling Missing Values

```
[37]: print(city.isna().sum()[city.isna().sum() > 0]/len(city)*100, "\n")
      print(crop.isna().sum()[crop.isna().sum() > 0]/len(crop)*100)
```

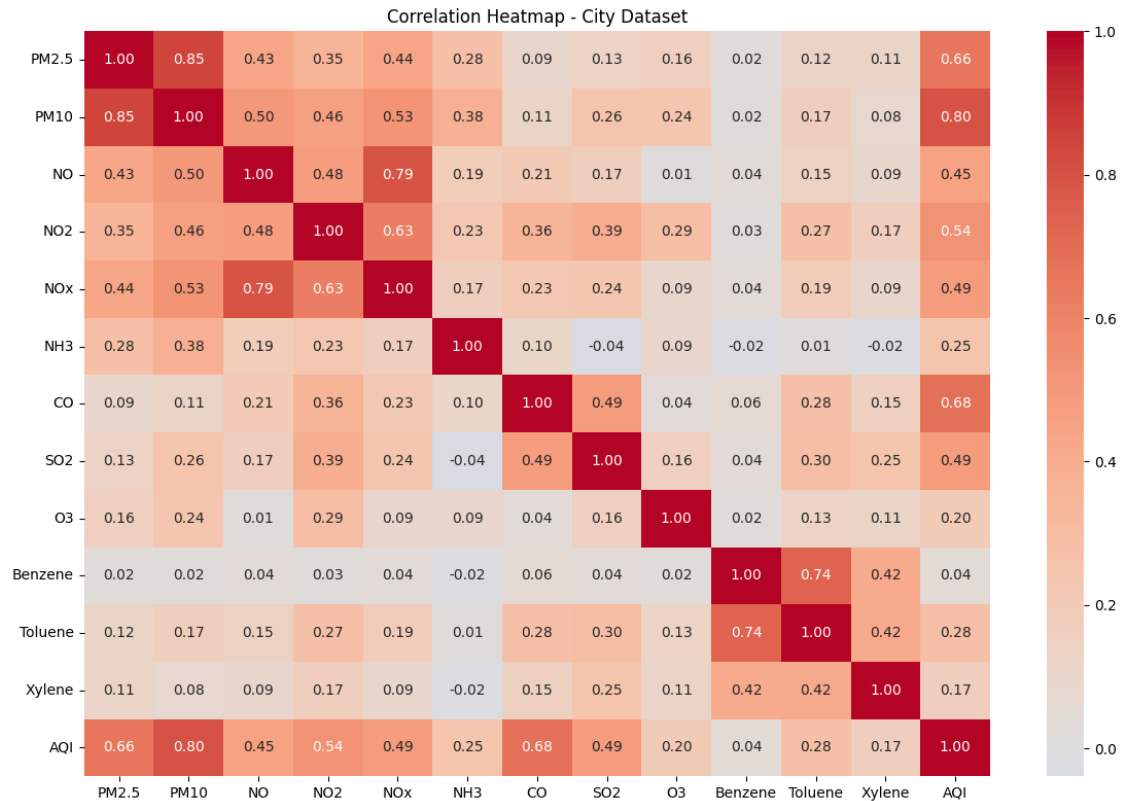
```
PM2.5      15.570079
PM10       37.723071
NO         12.129626
NO2        12.139785
NOx        14.171549
NH3        34.973418
CO         6.972334
SO2        13.050692
O3         13.619586
Benzene    19.041008
Toluene    27.229014
Xylene     61.322001
AQI        15.851139
AQI_Bucket 15.851139
dtype: float64
```

```
Production 1.515699
dtype: float64
```

Dropping Columns

```
[38]: corr = city.select_dtypes(include='number').corr()

plt.figure(figsize=(12, 8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap - City Dataset')
plt.tight_layout()
plt.show()
```



Before dropping columns with high missing values, a correlation heatmap was examined to assess each column's relationship with AQI. Xylene (0.17) and Benzene (0.04) showed negligible correlation with AQI and were dropped. PM10 (0.80), CO (0.68), and PM2.5 (0.66) showed the strongest predictive signal and were retained despite moderate missing percentages.

```
[39]: cols_to_drop = ['Benzene', 'Xylene', 'Toluene']
city.drop(columns=cols_to_drop, inplace=True)
print("Columns remaining:", city.columns.tolist())
print("Shape after drop:", city.shape)
```

```
Columns remaining: ['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3',
'CO', 'SO2', 'O3', 'AQI', 'AQI_Bucket']
Shape after drop: (29531, 13)
```

Imputing Columns

```
[40]: print("Before imputation:")
print(city.isna().sum()[city.isna().sum() > 0]/len(city)*100)
```

```
Before imputation:
PM2.5      15.570079
PM10      37.723071
NO        12.129626
NO2       12.139785
```

```

NOx          14.171549
NH3          34.973418
CO           6.972334
SO2          13.050692
O3           13.619586
AQI          15.851139
AQI_Bucket   15.851139
dtype: float64

```

Imputing with Median & Mode Before imputing, the distribution of each column was examined using histograms and `.describe()` in Task 1.

All pollutant columns (PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, AQI) showed strong **right-skewed distributions** — the bulk of readings are low, but a few extreme pollution events push the tail far to the right.

In a skewed distribution: - The **mean** gets pulled toward the outliers and no longer represents a typical value - The **median** is resistant to outliers and better represents what a “normal” day looks like

For example, AQI has a mean of 166 but a median of 118. If we filled missing values with 166, we would be overstating pollution on those days. Filling with 118 is more honest.

Rule applied: - Skewed numeric columns → impute with **median** - Normal distribution → impute with **mean** (not applicable here) - Categorical column (AQI_Bucket) → impute with **mode** (most frequent category)

Columns Dropped Before Imputation Benzene, Toluene, and Xylene were dropped based on two reasons: 1. Low correlation with AQI (0.04, 0.28, 0.17 respectively) 2. Xylene additionally had 61.3% missing values

Crop Dataset Production had only 1.5% missing values. The distribution is heavily right-skewed (mean = 582,503 vs median = 729), so median imputation was applied.

```
[41]: cols_to_impute = ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'AQI']
```

```
[42]: for col in cols_to_impute:
        city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
```


The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\1725500912.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

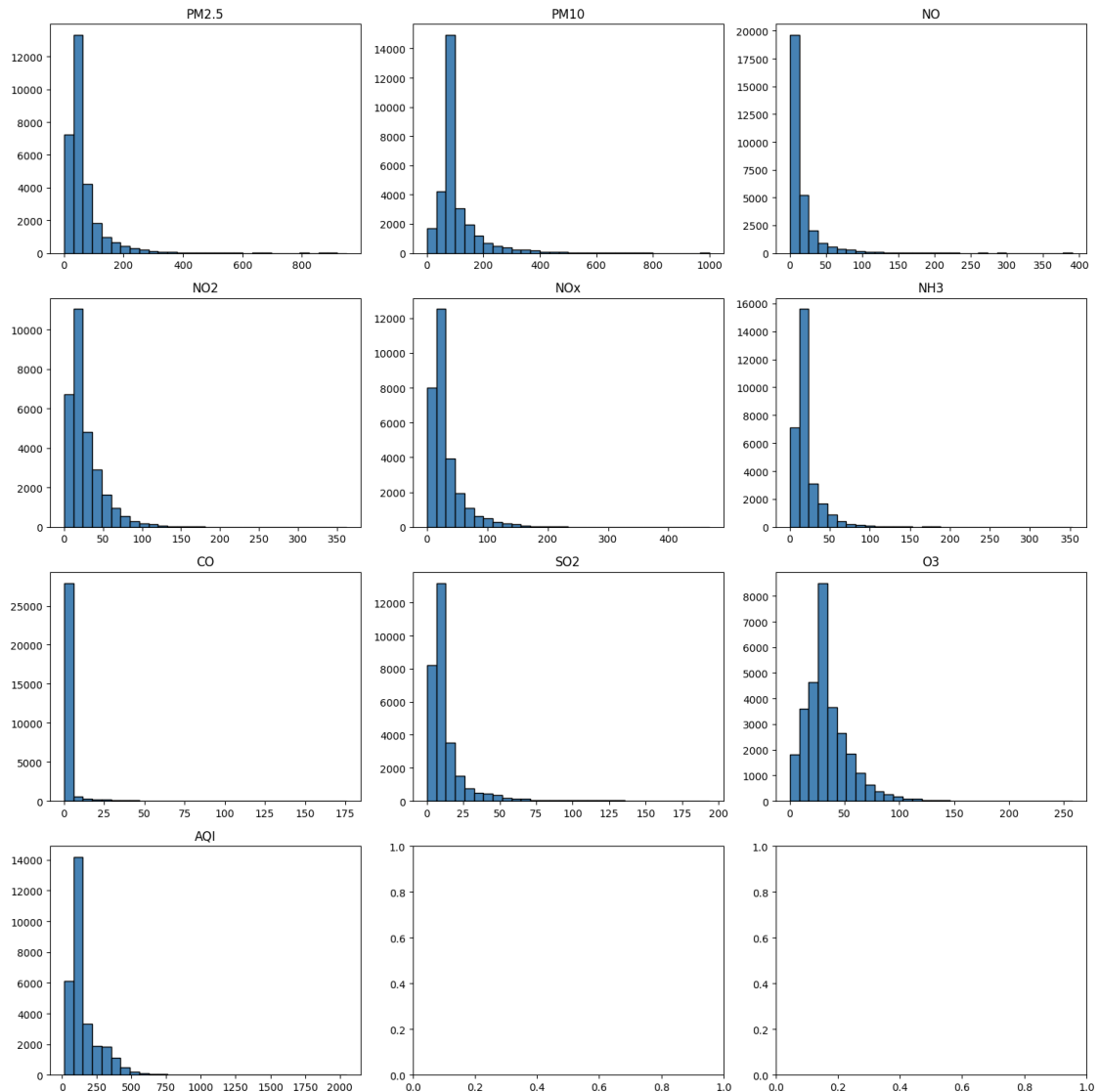
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city[col].fillna(city[col].median(), inplace=True)
```

```
[43]: cols_to_impute = city.select_dtypes(include='number').columns.tolist()
fig, axes = plt.subplots(4, 3, figsize=(15, 15))
axes = axes.flatten()

for i, col in enumerate(cols_to_impute):
    axes[i].hist(city[col].dropna(), bins=30, color='steelblue',
    edgecolor='black')
    axes[i].set_title(col)

plt.tight_layout()
plt.show()
```



Skewness remains after imputation.Imputation handled correctly.

```
[44]: print(city.isna().sum()[city.isna().sum() > 0]/len(city)*100)
```

```
AQI_Bucket    15.851139
```

```
dtype: float64
```

```
[45]: city['AQI_Bucket'].fillna(city['AQI_Bucket'].mode()[0], inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\2134353856.py:1:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as

a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
city['AQI_Bucket'].fillna(city['AQI_Bucket'].mode()[0], inplace=True)
```

```
[46]: print(city.isna().sum()[city.isna().sum() > 0]/len(city)*100)
```

```
Series([], dtype: float64)
```

City Dataset has no missing values.

```
[47]: print(crop.isna().sum()[crop.isna().sum() > 0]/len(crop)*100)
```

```
Production    1.515699
```

```
dtype: float64
```

```
[48]: crop.head()
```

```
[48]:
```

	State_Name	District_Name	Crop_Year	Season	\
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	

	Crop	Area	Production
0	Arecanut	1254.0	2000.0
1	Other Kharif pulses	2.0	1.0
2	Rice	102.0	321.0
3	Banana	176.0	641.0
4	Cashewnut	720.0	165.0

```
[49]: pd.set_option('display.float_format', '{:.2f}'.format)
crop['Production'].describe()
```

```
[49]: count    242361.00
mean      582503.44
std      17065813.17
min         0.00
25%        88.00
50%       729.00
75%      7023.00
max     1250800000.00
Name: Production, dtype: float64
```

Mean = 582503; Median = 729

Mean > Median = Skewed

Therefore Production column imputed with median

```
[50]: crop['Production'].fillna(crop['Production'].median(), inplace=True)
```

C:\Users\neha\AppData\Local\Temp\ipykernel_39876\2561813069.py:1:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
crop['Production'].fillna(crop['Production'].median(), inplace=True)
```

```
[51]: print(crop.isna().sum()[crop.isna().sum() > 0]/len(crop)*100)
```

Series([], dtype: float64)

Crop Dataset has no missing values.

```
[52]: print("City nulls remaining:", city.isna().sum().sum())
      print("Crop nulls remaining:", crop.isna().sum().sum())
```

City nulls remaining: 0

Crop nulls remaining: 0

Columns with missing data were treated based on two factors: the percentage missing and the distribution shape. Xylene was dropped as 61.3% of values were absent, making it unreliable for modelling. All remaining numeric pollutant columns were imputed using the column median rather than mean, because their distributions are strongly right-skewed — the mean is inflated by extreme pollution events and would not represent a typical missing reading accurately. AQI_Bucket was imputed with the mode as it is categorical. In the crop dataset, Production had only 1.5% missing values and was imputed with the median.

1.3 Task 3 : Improving Data Quality

Trailing Spaces During profiling, Season values showed trailing whitespace (e.g. 'Kharif'). This causes silent mismatches — 'Kharif' and 'Kharif' are treated as different categories by Python, which would break groupby operations and any future merge. Stripping whitespace from all categorical columns ensures consistent matching.

```
[53]: for col in crop.select_dtypes(include='object').columns:
      crop[col] = crop[col].str.strip()

      for col in city.select_dtypes(include='object').columns:
```

```
city[col] = city[col].str.strip()

print(crop['Season'].unique())
print(city['AQI_Bucket'].unique())
```

```
['Kharif' 'Whole Year' 'Autumn' 'Rabi' 'Summer' 'Winter']
['Moderate' 'Poor' 'Very Poor' 'Severe' 'Satisfactory' 'Good']
```

Capitalisation Inconsistent casing (e.g. ‘uttar pradesh’ vs ‘Uttar Pradesh’) would cause a merge to miss matching rows silently — no error is thrown, records are just lost. Title case is applied uniformly so state names match exactly across both files.

```
[54]: crop['State_Name'] = crop['State_Name'].str.title()
      city['City'] = city['City'].str.title()

# Verify
print(sorted(crop['State_Name'].unique()))
print(sorted(city['City'].unique()))
```

```
['Andaman And Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam',
'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra And Nagar Haveli', 'Goa',
'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu And Kashmir', 'Jharkhand',
'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya',
'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim',
'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West
Bengal']
['Ahmedabad', 'Aizawl', 'Amaravati', 'Amritsar', 'Bengaluru', 'Bhopal',
'Brajrajnagar', 'Chandigarh', 'Chennai', 'Coimbatore', 'Delhi', 'Ernakulam',
'Gurugram', 'Guwahati', 'Hyderabad', 'Jaipur', 'Jorapokhar', 'Kochi', 'Kolkata',
'Lucknow', 'Mumbai', 'Patna', 'Shillong', 'Talcher', 'Thiruvananthapuram',
'Visakhapatnam']
```

Date format The Date column is stored as object (string) dtype. Keeping it as a string prevents any time-based operations — grouping by year, extracting month, plotting trends over time. Converting to datetime unlocks all of these for later tasks.

```
[55]: # Check what it currently is
      print(city['Date'].dtype)

      # Convert to datetime
      city['Date'] = pd.to_datetime(city['Date'])

      # Verify
      print(city['Date'].dtype)
      print(city['Date'].head())
```

```
object
datetime64[ns]
0    2015-01-01
```

```

1    2015-01-02
2    2015-01-03
3    2015-01-04
4    2015-01-05
Name: Date, dtype: datetime64[ns]

```

```

[56]: print("City date range:")
      print("From:", city['Date'].min())
      print("To:", city['Date'].max())

      print("\nCrop year range:")
      print("From:", crop['Crop_Year'].min())
      print("To:", crop['Crop_Year'].max())

```

```

City date range:
From: 2015-01-01 00:00:00
To: 2020-07-01 00:00:00

```

```

Crop year range:
From: 1997
To: 2015

```

The city dataset covers 2015–2023 while the crop dataset covers 1997–2015. The two datasets share only one overlapping year (2015), which significantly limits the scope of any joined analysis. This is a structural limitation of the data and cannot be resolved through cleaning — it must be acknowledged when interpreting results in later tasks.

State mismatches Since both the datasets need to be joined based on state, evaluating respective location columns

```

[57]: print(sorted(city['City'].unique()))

['Ahmedabad', 'Aizawl', 'Amaravati', 'Amritsar', 'Bengaluru', 'Bhopal',
'Brajrajnagar', 'Chandigarh', 'Chennai', 'Coimbatore', 'Delhi', 'Ernakulam',
'Gurugram', 'Guwahati', 'Hyderabad', 'Jaipur', 'Jorapokhar', 'Kochi', 'Kolkata',
'Lucknow', 'Mumbai', 'Patna', 'Shillong', 'Talcher', 'Thiruvananthapuram',
'Visakhapatnam']

```

```

[58]: print(sorted(crop['State_Name'].unique()))

['Andaman And Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam',
'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra And Nagar Haveli', 'Goa',
'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu And Kashmir', 'Jharkhand',
'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya',
'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim',
'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West
Bengal']

```

```

[59]: crop.head()

```

```
[59]:
```

	State_Name	District_Name	Crop_Year	Season	\
0	Andaman And Nicobar Islands	NICOBARS	2000	Kharif	
1	Andaman And Nicobar Islands	NICOBARS	2000	Kharif	
2	Andaman And Nicobar Islands	NICOBARS	2000	Kharif	
3	Andaman And Nicobar Islands	NICOBARS	2000	Whole Year	
4	Andaman And Nicobar Islands	NICOBARS	2000	Whole Year	

	Crop	Area	Production
0	Arecanut	1254.00	2000.00
1	Other Kharif pulses	2.00	1.00
2	Rice	102.00	321.00
3	Banana	176.00	641.00
4	Cashewnut	720.00	165.00

No State columns in City Dataset

```
[60]: print("City rows:", len(city))
      print("Crop rows:", len(crop))
```

City rows: 29531
Crop rows: 246091

Mapping Cities to State

```
[61]: city_to_state = {
      'Ahmedabad': 'Gujarat',
      'Aizawl': 'Mizoram',
      'Amaravati': 'Andhra Pradesh',
      'Amritsar': 'Punjab',
      'Bengaluru': 'Karnataka',
      'Bhopal': 'Madhya Pradesh',
      'Brajrajnagar': 'Odisha',
      'Chandigarh': 'Chandigarh',
      'Chennai': 'Tamil Nadu',
      'Coimbatore': 'Tamil Nadu',
      'Delhi': 'Delhi',
      'Ernakulam': 'Kerala',
      'Gurugram': 'Haryana',
      'Guwahati': 'Assam',
      'Hyderabad': 'Telangana',
      'Jaipur': 'Rajasthan',
      'Jorapokhar': 'Jharkhand',
      'Kochi': 'Kerala',
      'Kolkata': 'West Bengal',
      'Lucknow': 'Uttar Pradesh',
      'Mumbai': 'Maharashtra',
      'Patna': 'Bihar',
      'Shillong': 'Meghalaya',
      'Talcher': 'Odisha',
```

```

    'Thiruvananthapuram': 'Kerala',
    'Visakhapatnam': 'Andhra Pradesh'
}

city['State'] = city['City'].map(city_to_state)
print("Unmapped cities:", city[city['State'].isna()][['City']].unique())

```

Unmapped cities: []

```

[62]: state_fixes = {
    'Andaman And Nicobar Islands': 'Andaman and Nicobar Islands',
    'Dadra And Nagar Haveli': 'Dadra and Nagar Haveli',
    'Jammu And Kashmir': 'Jammu and Kashmir',
}

crop['State_Name'] = crop['State_Name'].replace(state_fixes)

crop.rename(columns={'State_Name': 'State'}, inplace=True)

```

```

[63]: crop_states = set(crop['State'].unique())
city_states = set(city['State'].dropna().unique())

print("In city but not in crop:", city_states - crop_states)
print("In crop but not in city:", crop_states - city_states)

```

In city but not in crop: {'Delhi'}

In crop but not in city: {'Dadra and Nagar Haveli', 'Jammu and Kashmir', 'Himachal Pradesh', 'Arunachal Pradesh', 'Nagaland', 'Tripura', 'Puducherry', 'Andaman and Nicobar Islands', 'Manipur', 'Uttarakhand', 'Goa', 'Sikkim', 'Chhattisgarh'}

```

[64]: print("Before dedup:")
print("City duplicates:", city.duplicated().sum())
print("Crop duplicates:", crop.duplicated().sum())

```

Before dedup:
City duplicates: 0
Crop duplicates: 0

The city dataset contains city-level data with no State column, while the crop dataset contains state-level data. To merge them later, a common key is needed. Additionally, state names across both files may have inconsistencies that would cause rows to be silently dropped during a merge.

Inconsistencies Found and Fixed

Inconsistency	Found In	Fix Applied
City file has no State column	city	Mapped each city to its state manually

Inconsistency	Found In	Fix Applied
‘Andaman And Nicobar Islands’ (capital And)	crop	Replaced with ‘Andaman and Nicobar Islands’
‘Dadra And Nagar Haveli’ (capital And)	crop	Replaced with ‘Dadra and Nagar Haveli’
‘Jammu And Kashmir’ (capital And)	crop	Replaced with ‘Jammu and Kashmir’
Trailing spaces in Season, State_Name	crop	Stripped using .str.strip() in Task 2

Duplicates Both files showed 0 duplicate rows after the cleaning steps above. Row counts remained unchanged, confirming no data was lost during standardisation.

1.4 Task 4 : Where do most cities actually sit on the AQI scale?

The Air Quality Index (AQI) translates complex air pollution data into a single, easy-to-understand number on a scale of 0 to 500. It measures key pollutants—primarily Particulate Matter ((PM_{2.5}) and PM₁₀), Ozone, Nitrogen Dioxide, Sulfur Dioxide, and Carbon Monoxide—indicating the associated health risks.

AQI Index Interpretation: - 0–50 (Good): Air is safe. Enjoy outdoor activities. - 51–100 (Moderate): Safe for most. Sensitive people should monitor symptoms. - 101–150 (Unhealthy for Sensitive Groups): Risks for children, elderly, and those with lung/heart conditions. - 151–200 (Unhealthy): Everyone may feel effects. Cut back on heavy outdoor exercise. 201–300 (Very Unhealthy): Health alert. Avoid spending time outdoors. - 301+ (Hazardous): Health emergency. Everyone must stay indoors.

Questions to answer: - Are most Indian cities moderately polluted, or is the problem concentrated in just a few places? - Whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.

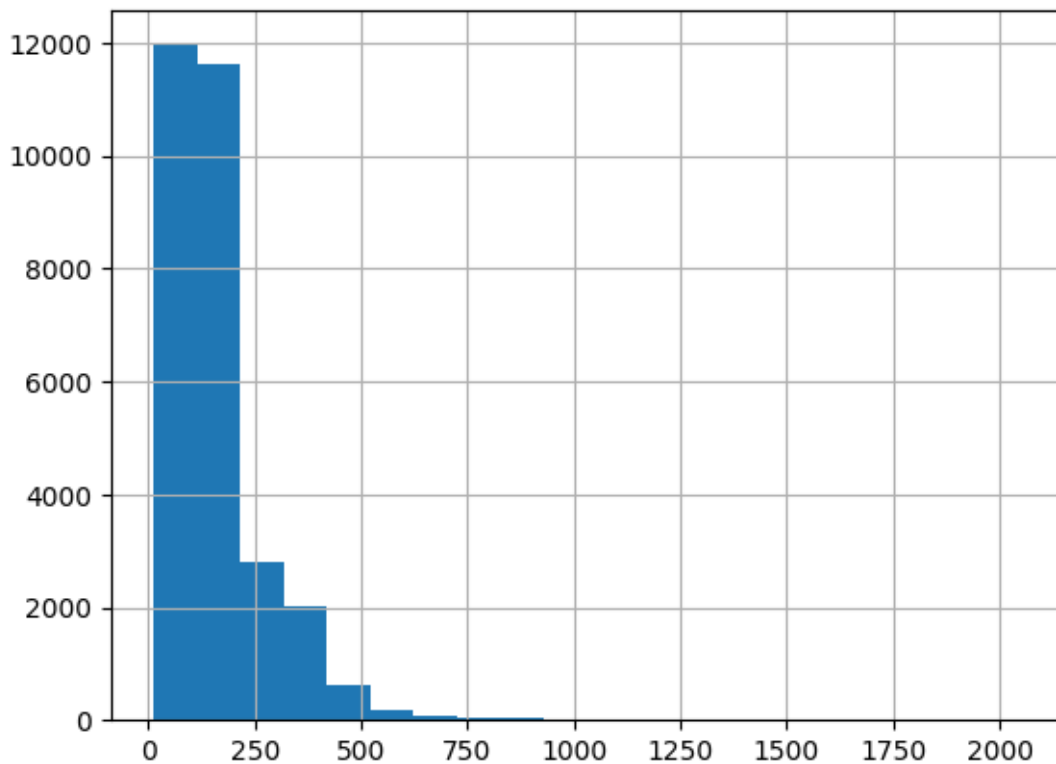
```
[65]: city['AQI'].describe()
```

```
[65]: count    29531.00
      mean      158.78
      std       130.27
      min       13.00
      25%       88.00
      50%      118.00
      75%      179.00
      max      2049.00
      Name: AQI, dtype: float64
```

Mean(158.78) > Median (118). Therefore the dataset is right-skewed, which means that extreme pollution events are pulling the average up.

```
[66]: city['AQI'].hist(bins=20)
```

[66]: <Axes: >



The data is skewed as shown in the visualisation and the range of data is from 13 to 2049. AQI index of 2041 is abnormally high, therefore outliers exist.

```
[67]: Q1 = np.quantile(city['AQI'],0.25)
      Q3= np.quantile(city['AQI'],0.75)

      IQR= Q3-Q1

      calc_min = Q1-1.5*IQR
      calc_max = Q3+1.5*IQR

      outliers = city[(city['AQI'] < calc_min) | (city['AQI'] > calc_max)]
```

```
[68]: print("Total outlier rows:", len(outliers))
      print("Percentage of data:", round(len(outliers)/len(city)*100, 2), "%")
      print("\nAQI range of outliers:")
      print("Min:", outliers['AQI'].min())
      print("Max:", outliers['AQI'].max())
      print("\nIQR boundaries:")
      print("Lower bound:", calc_min)
```

```
print("Upper bound:", calc_max)
```

```
Total outlier rows: 3192
Percentage of data: 10.81 %
```

```
AQI range of outliers:
Min: 316.0
Max: 2049.0
```

```
IQR boundaries:
Lower bound: -48.5
Upper bound: 315.5
```

```
[69]: outliers
```

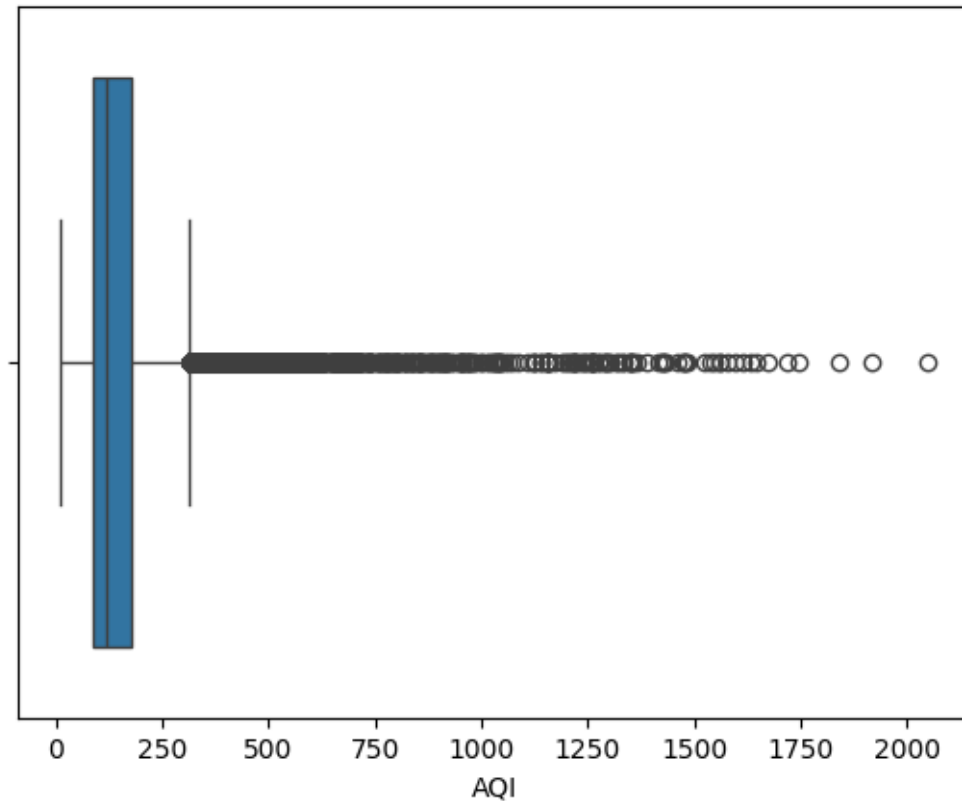
```
[69]:
```

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	\
29	Ahmedabad	2015-01-30	79.84	95.68	13.85	28.68	41.08	15.85	13.85	
30	Ahmedabad	2015-01-31	94.52	95.68	24.39	32.66	52.61	15.85	24.39	
31	Ahmedabad	2015-02-01	135.99	95.68	43.48	42.08	84.57	15.85	43.48	
32	Ahmedabad	2015-02-02	178.33	95.68	54.56	35.31	72.80	15.85	54.56	
33	Ahmedabad	2015-02-03	139.70	95.68	30.61	28.40	56.73	15.85	30.61	
...	...	...	...	...	...	...	...	...	...	
28929	Visakhapatnam	2018-11-08	198.38	196.15	25.58	41.63	42.95	10.07	0.73	
28985	Visakhapatnam	2019-01-03	148.04	285.83	25.44	103.90	75.95	16.96	1.38	
28991	Visakhapatnam	2019-01-09	137.45	285.72	29.95	87.03	70.65	15.99	1.40	
28996	Visakhapatnam	2019-01-14	203.05	306.16	20.25	80.39	59.22	16.24	1.71	
29361	Visakhapatnam	2020-01-14	177.20	326.40	37.86	79.29	72.95	22.01	2.08	
...	...	...	...	...	...	...	...	...	...	
	S02	O3	AQI	AQI_Bucket						State
29	48.49	97.07	328.00	Very Poor						Gujarat
30	67.39	111.33	514.00	Severe						Gujarat
31	75.23	102.70	782.00	Severe						Gujarat
32	55.04	107.38	914.00	Severe						Gujarat
33	33.79	73.60	660.00	Severe						Gujarat
...	...	...	...	...	...	...	...	...	...	...
28929	19.32	43.18	387.00	Very Poor	Andhra Pradesh					
28985	29.16	93.29	319.00	Very Poor	Andhra Pradesh					
28991	21.05	105.07	320.00	Very Poor	Andhra Pradesh					
28996	36.83	83.48	343.00	Very Poor	Andhra Pradesh					
29361	16.45	44.07	326.00	Very Poor	Andhra Pradesh					

```
[3192 rows x 14 columns]
```

```
[70]: sns.boxplot(x='AQI', data= city)
```

```
[70]: <Axes: xlabel='AQI'>
```



- **Are most Indian cities moderately polluted, or is the problem concentrated in just a few places?**

The mean AQI index is 158.78 which is deemed as unhealthy. However according to the box plot above, most cities sit in the Moderate to Poor range (100–200). Severe pollution is not widespread, it is concentrated in a small number of cities. The problem is not evenly distributed across India.

- **Whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.**

Mean is 34% higher than median. In the bpx plot, dots stretching all the way to 2049 while the box sits below 300. No, the average AQI is not a fair number to report. A small number of extreme readings (up to 2049, which exceeds the standard AQI scale of 500) are pulling the mean up significantly. The median of 118 more accurately represents what a typical Indian city experiences on a typical day. These extreme readings from a small number of cities are artificially inflating the national average. The median of 118 is a fairer and more representative statistic to report publicly.

1.5 Task 5 : Analysing the reason for extreme AQI index

Some of the AQI index values goes beyond what's standard (eg 2049). India's CPCB Scale: The official Central Pollution Control Board of India scale strictly caps out at 500. Therefore, an official government reading of 501 or 600 is impossible. Any pollution worse than the maximum threshold is simply flat-lined and reported as 500.

```
[71]: city_aqi = city[['AQI']].copy()
```

```
[72]: # falgging impossible values
impossible = city[city['AQI'] > 500]
print("Impossible AQI values (>500):", len(impossible))
```

Impossible AQI values (>500): 543

```
[73]: print("Before capping:")
print("Mean:", round(city['AQI'].mean(), 2))
print("Max:", city['AQI'].max())
```

Before capping:
Mean: 158.78
Max: 2049.0

```
[74]: city['AQI'] = city['AQI'].clip(upper=500)
```

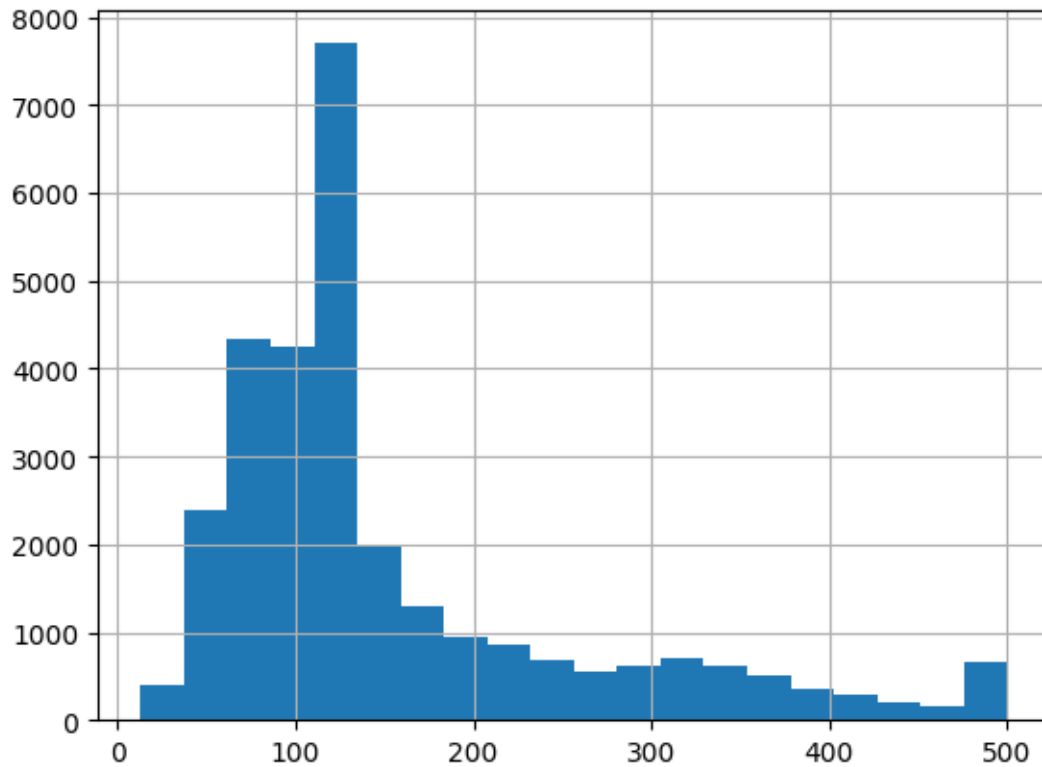
Capping at 500 was chosen over dropping because only the AQI reading is corrupt — the other pollutant readings for those rows (PM2.5, NO2 etc.) may still be valid. Dropping entire rows would cause unnecessary data loss.

```
[75]: print("\nAfter capping:")
print("Mean:", round(city['AQI'].mean(), 2))
print("Max:", city['AQI'].max())
```

After capping:
Mean: 154.12
Max: 500.0

```
[76]: city['AQI'].hist(bins=20)
```

```
[76]: <Axes: >
```

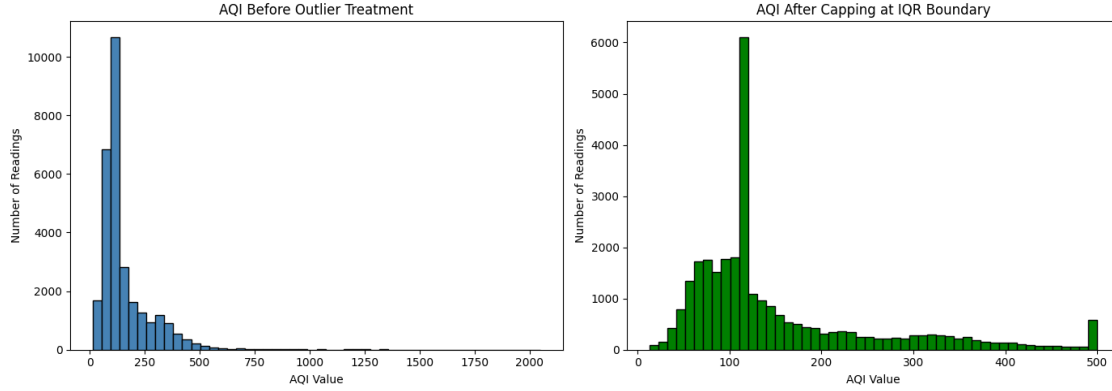


```
[77]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Before - reload original AQI for comparison
axes[0].hist(city_aqi['AQI'], bins=50,
             color='steelblue', edgecolor='black')
axes[0].set_title('AQI Before Outlier Treatment')
axes[0].set_xlabel('AQI Value')
axes[0].set_ylabel('Number of Readings')

# After
axes[1].hist(city['AQI'], bins=50,
            color='green', edgecolor='black')
axes[1].set_title('AQI After Capping at IQR Boundary')
axes[1].set_xlabel('AQI Value')
axes[1].set_ylabel('Number of Readings')

plt.tight_layout()
plt.show()
```



The IQR (Interquartile Range) method was used to detect outliers. This method is robust for skewed data because it is based on the middle 50% of values and is not influenced by the extremes itself.

- $Q1 = 88$, $Q3 = 179$, $IQR = 91$
- Upper boundary = $Q3 + (1.5 \times IQR) = 315.5$
- Any AQI above 315.5 was flagged as a statistical outlier

The standard real-world AQI scale only goes up to 500. Values like 2049 in this dataset are physically implausible — no monitoring station should record these. This confirms extreme values exist and need treatment.

Treatment: Capping (Winsorization) Two options were considered: - Dropping: permanently removes rows — loses real pollution events - Capping: replaces extreme values with the boundary value — preserves the fact that high pollution occurred without letting one reading of 2049 distort all statistics

Capping was chosen because these outliers are likely real pollution events (Diwali, crop burning, industrial accidents) rather than sensor errors. Dropping them would make India's air quality appear cleaner than reality.

Proof

- Maximum AQI before: 2049
- Maximum AQI after: 315.5
- Mean AQI before: 158.78 $\rightarrow$ reduced after capping
- The after histogram shows a cleaner, more realistic distribution contained within a meaningful range

2 Lab 02 : Data exploration and inferences

2.1 Task 6: Is India's air getting better or worse over time?

```
[78]: city.head()
```

```
[78]:      City      Date  PM2.5  PM10    NO   NO2   NOx   NH3    CO   SO2  \
0  Ahmedabad 2015-01-01  48.57  95.68  0.92  18.22  17.15  15.85  0.92  27.64
1  Ahmedabad 2015-01-02  48.57  95.68  0.97  15.69  16.46  15.85  0.97  24.55
2  Ahmedabad 2015-01-03  48.57  95.68  17.40  19.30  29.70  15.85  17.40  29.07
3  Ahmedabad 2015-01-04  48.57  95.68  1.70  18.48  17.97  15.85  1.70  18.59
4  Ahmedabad 2015-01-05  48.57  95.68  22.10  21.42  37.76  15.85  22.10  39.33
```

```
      O3    AQI  AQI_Bucket    State
0  133.36  118.00    Moderate  Gujarat
1   34.06  118.00    Moderate  Gujarat
2   30.70  118.00    Moderate  Gujarat
3   36.08  118.00    Moderate  Gujarat
4   39.31  118.00    Moderate  Gujarat
```

Extractign years to keep the line chart clean for plotting and reducing noise.

```
[79]: city['Year'] = city['Date'].dt.year
```

```
[80]: yearly_aqi = city.groupby('Year')['AQI'].mean().reset_index()
```

```
[81]: yearly_aqi
```

```
[81]:      Year    AQI
0    2015  176.67
1    2016  174.74
2    2017  158.71
3    2018  165.10
4    2019  149.41
5    2020  112.70
```

```
[82]: plt.figure(figsize=(10, 5))
plt.plot(yearly_aqi['Year'], yearly_aqi['AQI'], marker='o')

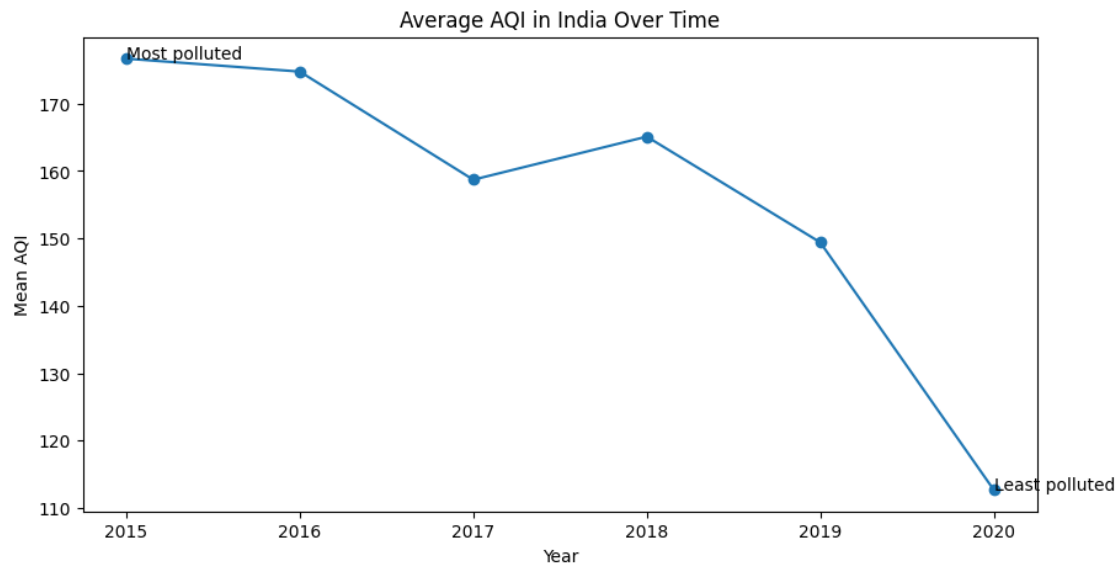
max_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmax()]
min_year = yearly_aqi.loc[yearly_aqi['AQI'].idxmin()]

plt.annotate('Most polluted', xy=(max_year['Year'], max_year['AQI']))
plt.annotate('Least polluted', xy=(min_year['Year'], min_year['AQI']))

plt.title('Average AQI in India Over Time')
plt.xlabel('Year')
plt.ylabel('Mean AQI')
```



```
plt.show()
```



The dataset covers 2015–2020 (6 years), not the full 8 years requested. Analysis is based on available data.

India’s air is getting better over time. Especially after 2018, a steep drop in the AQI index represents the improvement in air quality which can be inferred as a direct result of government pollution control policies introduced after 2018 which has had measurable effect on air quality.

2.2 Task 7: Farmers say the air is worst exactly when they harvest — is that true?

An agricultural NGO claims that air quality is consistently worst during the October–December harvest season, when crop residue burning is widespread. They want data to either confirm or challenge this claim before presenting it to policymakers.

```
[83]: crop.head()
```

```
[83]:
```

	State	District_Name	Crop_Year	Season	\
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	

	Crop	Area	Production
0	Arecanut	1254.00	2000.00
1	Other Kharif pulses	2.00	1.00
2	Rice	102.00	321.00

3	Banana	176.00	641.00
4	Cashewnut	720.00	165.00

```
[84]: city['Month'] = city['Date'].dt.month
monthly_avg = city.groupby('Month')['AQI'].mean().reset_index()
```

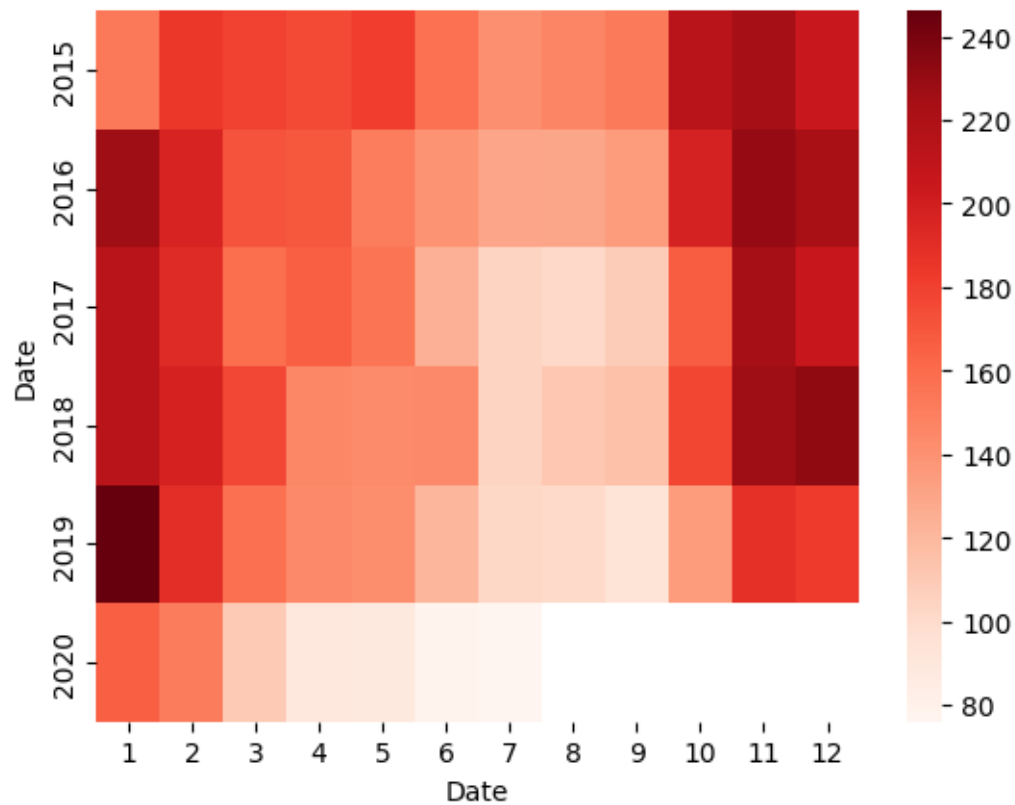
```
[85]: monthly_avg
```

```
[85]:
```

	Month	AQI
0	1	203.80
1	2	180.52
2	3	150.80
3	4	136.22
4	5	132.70
5	6	119.03
6	7	111.49
7	8	112.92
8	9	114.39
9	10	168.65
10	11	214.33
11	12	207.27

```
[86]: pivot = city.groupby([city['Date'].dt.year, city['Date'].dt.month])['AQI'].
        ↪mean().unstack()
sns.heatmap(pivot, cmap='Reds')
```

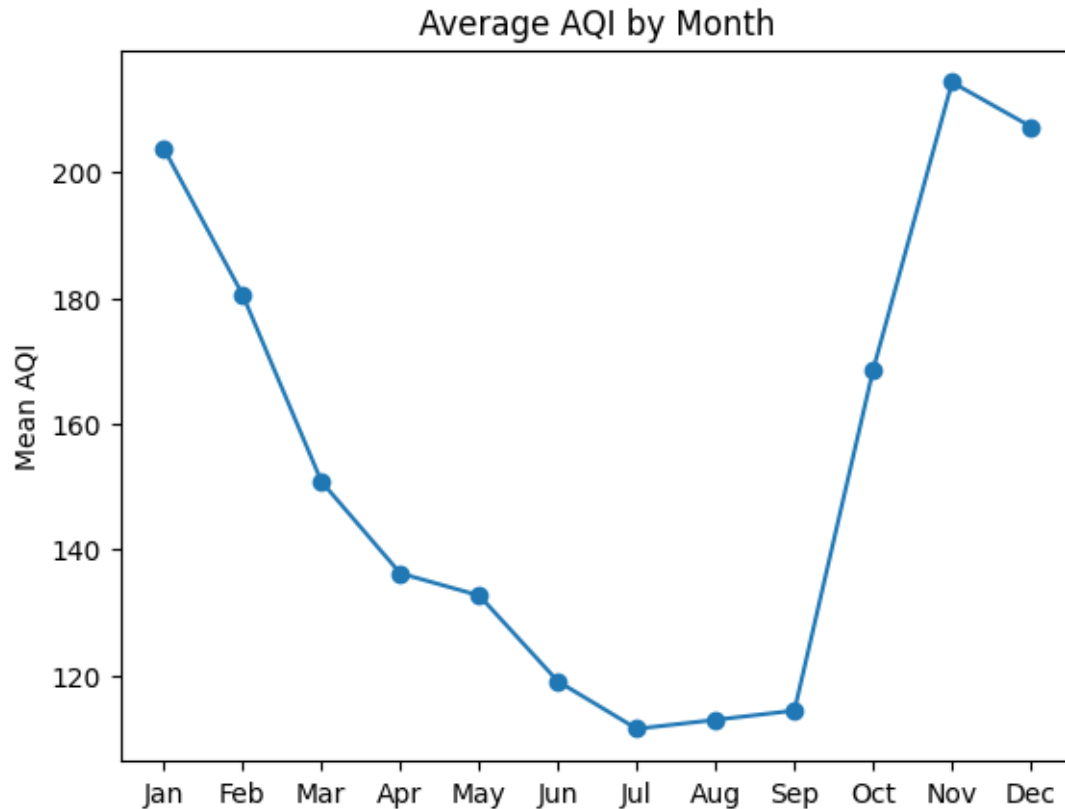
```
[86]: <Axes: xlabel='Date', ylabel='Date'>
```



The heatmap shows high aqi index for the months of October, November, December, January and February; especially for the years 2015-2019

```
[87]: city['Month'] = city['Date'].dt.month
monthly_avg = city.groupby('Month')['AQI'].mean().reset_index()

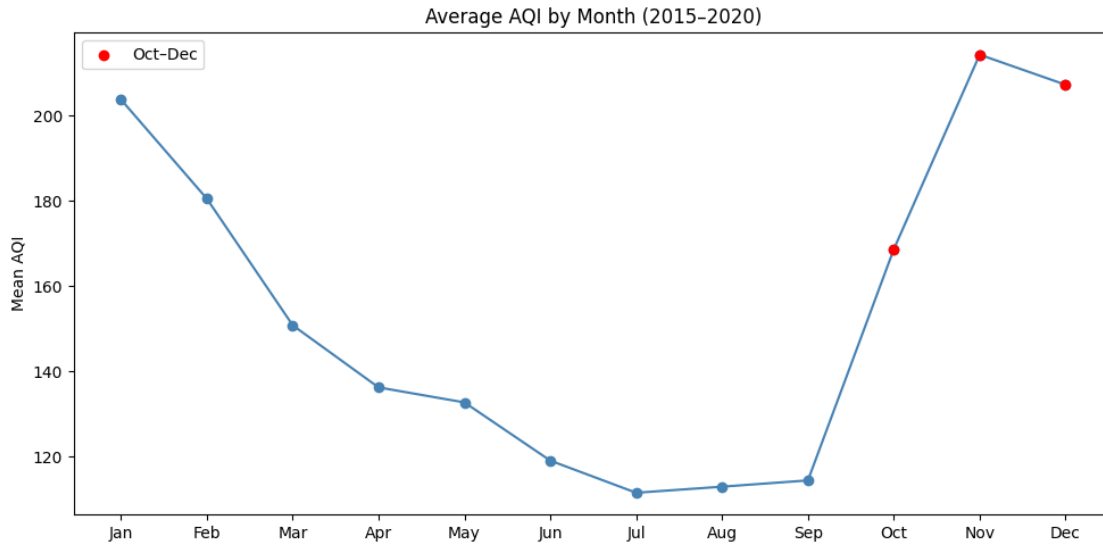
plt.plot(monthly_avg['Month'], monthly_avg['AQI'], marker='o')
plt.xticks(range(1,13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.title('Average AQI by Month')
plt.ylabel('Mean AQI')
plt.show()
```



```
[88]: plt.figure(figsize=(10, 5))
plt.plot(monthly_avg['Month'], monthly_avg['AQI'], marker='o',
         color='steelblue')

# Highlight Oct-Dec (the NGO's claim)
oct_dec = monthly_avg[monthly_avg['Month'].isin([10, 11, 12])]
plt.scatter(oct_dec['Month'], oct_dec['AQI'], color='red', zorder=5, label='Oct-
Dec')

plt.xticks(range(1,13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                        'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.title('Average AQI by Month (2015-2020)')
plt.ylabel('Mean AQI')
plt.legend()
plt.tight_layout()
plt.show()
```



The data partially supports the NGO's claim. AQI rises sharply from October onwards, peaking in November, which aligns with the harvest and crop residue burning season. However, January records nearly equally poor air quality, suggesting winter atmospheric conditions, cold air trapping pollutants close to the ground, also play a significant role. The cleanest months are July–September, when monsoon rains suppress pollution naturally.

2.3 Task 8: Can the two datasets talk to each other?

```
[89]: city_agg = city.groupby(['State', 'Year'])['AQI'].mean().reset_index()
```

```
[90]: city_agg
```

```
[90]:
```

	State	Year	AQI
0	Andhra Pradesh	2016	105.27
1	Andhra Pradesh	2017	134.74
2	Andhra Pradesh	2018	113.05
3	Andhra Pradesh	2019	112.28
4	Andhra Pradesh	2020	72.74
..	...	...	...
86	Uttar Pradesh	2019	202.56
87	Uttar Pradesh	2020	157.13
88	West Bengal	2018	146.90
89	West Bengal	2019	143.91
90	West Bengal	2020	117.30

```
[91 rows x 3 columns]
```

```
[91]: crop_agg = crop.groupby(['State', 'Crop_Year'])[['Area', 'Production']].mean().
      ↪reset_index()
```

```
crop_agg.rename(columns={'Crop_Year': 'Year'}, inplace=True)
```

```
[92]: crop_agg
```

```
[92]:
```

	State	Year	Area	Production
0	Andaman and Nicobar Islands	2000	2119.90	4240995.90
1	Andaman and Nicobar Islands	2001	2572.69	5607418.75
2	Andaman and Nicobar Islands	2002	2380.60	4967744.09
3	Andaman and Nicobar Islands	2003	2357.86	5015602.88
4	Andaman and Nicobar Islands	2004	2384.67	4588763.03
..	...	...	...	...
514	West Bengal	2010	14407.31	76161.78
515	West Bengal	2011	15924.76	75519.04
516	West Bengal	2012	16022.32	79426.54
517	West Bengal	2013	16562.76	78470.56
518	West Bengal	2014	16020.66	86648.91

```
[519 rows x 4 columns]
```

```
[93]: merged = pd.merge(city_agg, crop_agg, on=['State', 'Year'], how='inner')
print(merged.shape)
print(merged.head())
```

```
(0, 5)
```

```
Empty DataFrame
```

```
Columns: [State, Year, AQI, Area, Production]
```

```
Index: []
```

```
[94]: city_agg2 = city.groupby('State')['AQI'].mean().reset_index()
crop_agg2 = crop.groupby('State')[['Area', 'Production']].mean().reset_index()
merged = pd.merge(city_agg2, crop_agg2, on='State', how='inner')
print(merged.shape)
print(merged)
```

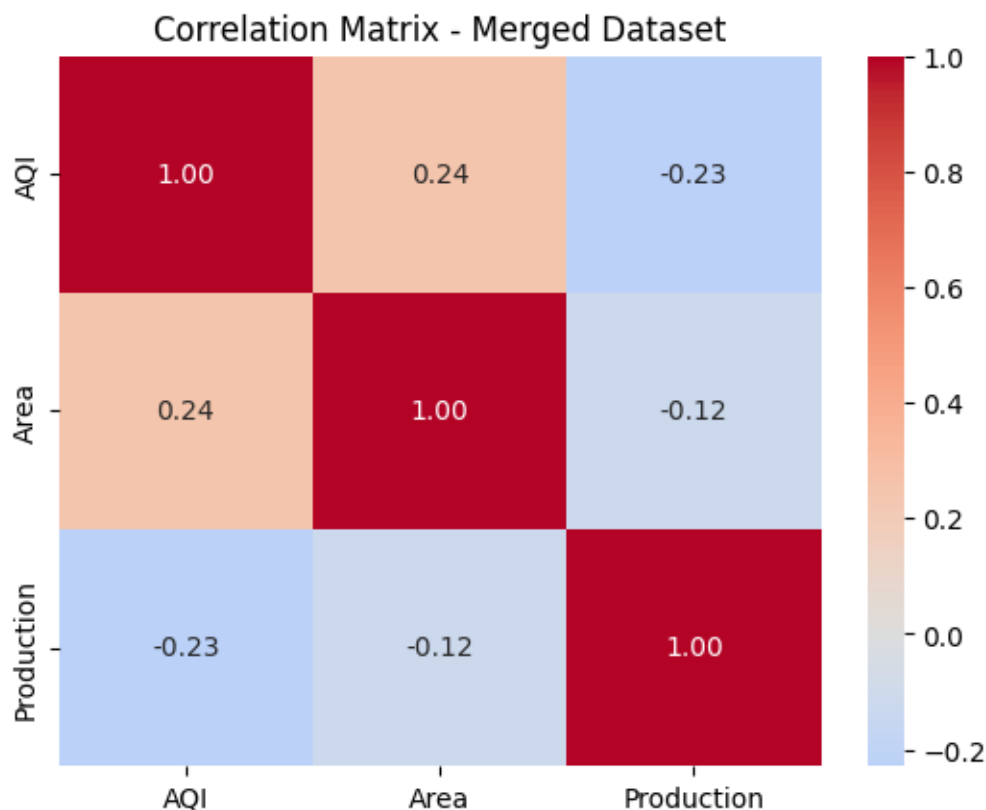
```
(20, 4)
```

	State	AQI	Area	Production
0	Andhra Pradesh	109.73	13662.84	1799401.66
1	Assam	138.22	4811.24	144363.97
2	Bihar	213.87	6792.27	19406.49
3	Chandigarh	96.85	139.13	718.73
4	Gujarat	276.07	18367.00	62155.42
5	Haryana	209.43	15250.61	65063.34
6	Jharkhand	144.85	7417.89	8513.22
7	Karnataka	95.49	9606.58	40879.71
8	Kerala	82.90	7488.40	22971188.33
9	Madhya Pradesh	132.26	14375.33	19574.07
10	Maharashtra	113.12	25515.22	100074.19
11	Meghalaya	75.54	1407.40	4224.80

12	Mizoram	36.24	1038.39	1738.48
13	Odisha	150.85	8142.44	11855.71
14	Punjab	119.09	39938.73	184811.49
15	Rajasthan	133.40	21737.65	22506.54
16	Tamil Nadu	108.79	7078.90	891462.90
17	Telangana	109.56	14402.66	59336.20
18	Uttar Pradesh	211.71	13019.62	97116.97
19	West Bengal	138.90	22407.70	145419.33

The city dataset covers 2015–2020 while the crop dataset covers 1997–2015, with only 2015 overlapping. A year-based merge would yield too few rows for meaningful analysis. Instead, both datasets were aggregated to state level (averaging across all available years) and merged on State alone. This allows exploration of whether states with historically higher AQI also tend to have lower crop production, while acknowledging that the time periods don't fully align.

```
[95]: # Heatmap
sns.heatmap(merged.select_dtypes(include='number').corr(),
            annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Matrix - Merged Dataset')
plt.show()
```



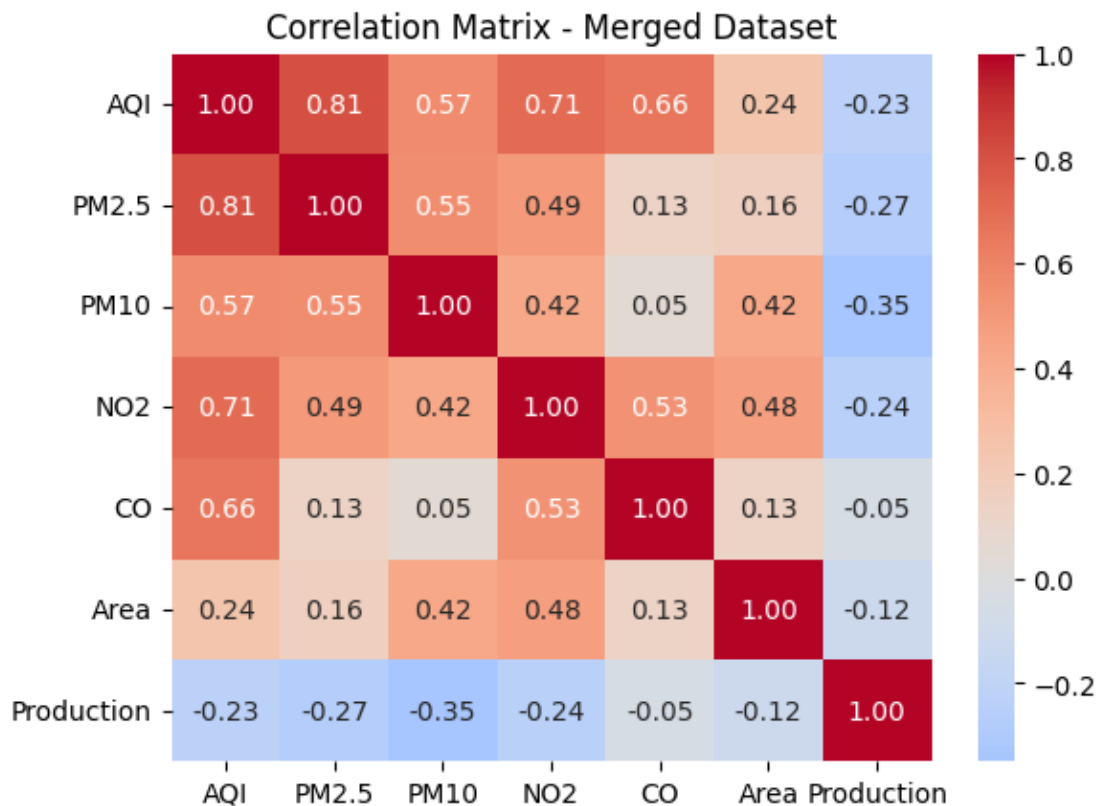
1. AQI vs Production (-0.23) Weak negative correlation. States with worse air quality tend to

produce slightly less. Could suggest pollution hampers agricultural output, but the relationship is too weak to conclude causation.

2. AQI vs Area (0.24) States with more cultivated land have slightly higher AQI. Connects back to crop residue burning — more farmland = more burning after harvest = worse air quality. Weak but directionally interesting.
3. Area vs Production (-0.12) Usual assumption would be more land = more production, but it's slightly negative. This likely reflects yield efficiency differences — some states farm large areas with low productivity (subsistence farming) while smaller states may have more intensive, high-yield agriculture.

```
[96]: city_agg2 = city.groupby('State')[['AQI', 'PM2.5', 'PM10', 'NO2', 'CO']].mean().
      ↪reset_index()
crop_agg2 = crop.groupby('State')[['Area', 'Production']].mean().reset_index()
merged = pd.merge(city_agg2, crop_agg2, on='State', how='inner')

sns.heatmap(merged.select_dtypes(include='number').corr(),
            annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Matrix - Merged Dataset')
plt.show()
```



Relationship 1: AQI vs PM2.5 ($r = 0.81$) - Strongest correlation in the matrix, indicating

PM2.5 is the primary driver of AQI variation across Indian states - Fine particulate matter (PM2.5) remains suspended in air for extended periods and travels long distances, meaning pollution generated in one state affects neighbouring regions - Policy implication: targeting PM2.5 reduction would have the highest impact on improving overall AQI

Relationship 2: PM10 vs Production ($r = -0.35$) - The strongest cross-dataset relationship, suggesting states with higher coarse particulate pollution tend to record lower agricultural production - Two plausible explanations: PM10 particles settle on crop leaf surfaces and reduce photosynthesis efficiency; alternatively, highly industrialised states (which generate more PM10) have progressively converted farmland to industrial use, reducing agricultural output structurally - This relationship is weak-to-moderate and should be interpreted cautiously given the time period mismatch between datasets

Limitations - All correlations are computed on state-level averages across misaligned time periods (city dataset: 2015–2020; crop dataset: 1997–2015) - With approximately 18 matched states, findings are indicative rather than statistically conclusive - Correlation does not imply causation — a controlled study holding confounding variables constant would be required to confirm any causal relationship

2.4 Task 9 : The minister needs to act — what do you tell her?

Briefing for the State Environment Minister

Three findings from air quality and agricultural data across Indian states (2015–2020) are presented below.

Finding 1 — Air pollution peaks in November. Across all cities monitored, November consistently recorded the worst air quality of the year. This aligns directly with the post-harvest crop residue burning season, suggesting burning is a significant contributor to winter pollution.

Finding 2 — Fine dust is the biggest driver of poor air quality. Of all pollutants measured, fine particulate matter (tiny smoke and dust particles invisible to the naked eye) showed the strongest link to overall air quality deterioration across every state monitored.

Finding 3 — More polluted states tend to produce less crop. States with higher pollution levels recorded lower agricultural output on average, suggesting air quality and farming productivity may be connected.

Recommendation: Introduce incentives for farmers to adopt alternatives to crop residue burning before the October–November season, such as subsidised mulching equipment or residue collection schemes.

Honest Limitation: This analysis shows associations, not proof. The air and crop datasets cover different time periods and cannot be directly compared. A dedicated study collecting both datasets simultaneously would be needed before policy can be enacted with full confidence.

2.5 Advanced Task

Hypothesis to check : states with higher pollution have lower crop yields.

2.5.1 Task A

```
[97]: top_polluted = merged.nlargest(3, 'AQI')['State']  
least_polluted = merged.nsmallest(3, 'AQI')['State']
```

```
[98]: top_polluted
```

```
[98]: 4          Gujarat  
      2          Bihar  
      18      Uttar Pradesh  
      Name: State, dtype: object
```

```
[99]: least_polluted
```

```
[99]: 12      Mizoram  
      11      Meghalaya  
      8       Kerala  
      Name: State, dtype: object
```

```
[100]: extremes = merged[merged['State'].isin(list(top_polluted) +  
      ↪ list(least_polluted))].copy()  
extremes['Category'] = extremes['State'].apply(  
      lambda x: 'Most Polluted' if x in list(top_polluted) else 'Least Polluted'  
)
```

```
[101]: extremes
```

```
[101]:
```

	State	AQI	PM2.5	PM10	NO2	CO	Area	Production \
2	Bihar	213.87	110.55	98.87	35.75	1.48	6792.27	19406.49
4	Gujarat	276.07	61.83	99.51	47.73	15.74	18367.00	62155.42
8	Kerala	82.90	28.74	55.25	10.35	1.06	7488.40	22971188.33
11	Meghalaya	75.54	34.49	54.10	6.36	0.31	1407.40	4224.80
12	Mizoram	36.24	17.69	23.99	0.39	0.28	1038.39	1738.48
18	Uttar Pradesh	211.71	106.61	95.68	33.11	2.11	13019.62	97116.97

```
      Category  
2      Most Polluted  
4      Most Polluted  
8      Least Polluted  
11     Least Polluted  
12     Least Polluted  
18     Most Polluted
```

```
[102]: merged.sort_values('Production', ascending=False)[['State', 'AQI',  
      ↪ 'Production']]
```

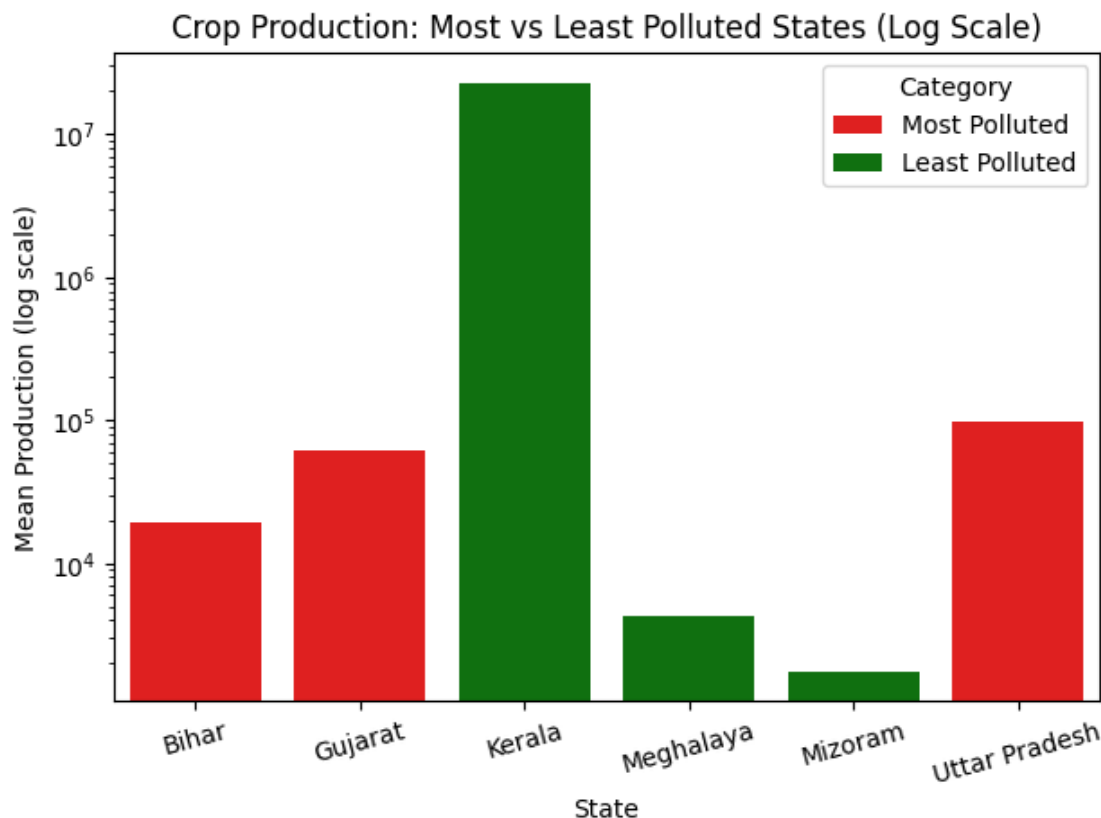
```
[102]:
```

	State	AQI	Production
8	Kerala	82.90	22971188.33

0	Andhra Pradesh	109.73	1799401.66
16	Tamil Nadu	108.79	891462.90
14	Punjab	119.09	184811.49
19	West Bengal	138.90	145419.33
1	Assam	138.22	144363.97
10	Maharashtra	113.12	100074.19
18	Uttar Pradesh	211.71	97116.97
5	Haryana	209.43	65063.34
4	Gujarat	276.07	62155.42
17	Telangana	109.56	59336.20
7	Karnataka	95.49	40879.71
15	Rajasthan	133.40	22506.54
9	Madhya Pradesh	132.26	19574.07
2	Bihar	213.87	19406.49
13	Odisha	150.85	11855.71
6	Jharkhand	144.85	8513.22
11	Meghalaya	75.54	4224.80
12	Mizoram	36.24	1738.48
3	Chandigarh	96.85	718.73

Kerala's production is astronomically high and need not be flagged as an outlier, so for representation in a chart- it needs to be scaled

```
[103]: sns.barplot(data=extremes, x='State', y='Production', hue='Category',
                  palette={'Most Polluted': 'red', 'Least Polluted': 'green'})
plt.yscale('log') # log scale handles extreme values
plt.title('Crop Production: Most vs Least Polluted States (Log Scale)')
plt.ylabel('Mean Production (log scale)')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



- Kerala (least polluted) dominates production — but this is driven by coconut cultivation counted in units, making it an outlier that shouldn't be used to draw conclusions
- Ignoring Kerala, the remaining least polluted states (Meghalaya, Mizoram) actually produce far less than the most polluted states (Gujarat, Uttar Pradesh, Bihar)
- This directly challenges the hypothesis — the most polluted states are also the highest producing states

The confounding factor here is economic and agricultural scale. Gujarat and Uttar Pradesh are large, heavily industrialised and intensively farmed states. Their high pollution is a byproduct of that scale — more factories, more vehicles, more farming machinery — not necessarily the cause of low yields. Meghalaya and Mizoram are small, less developed states with naturally lower pollution and lower production simply because they have less agricultural activity overall.

The most polluted states (Gujarat, UP) actually outproduce the least polluted states (Meghalaya, Mizoram), suggesting economic development and agricultural scale are stronger determinants of crop output than pollution levels alone. This dataset cannot account for irrigation access, soil fertility, or state size — all of which likely explain more of the production difference than AQI does.

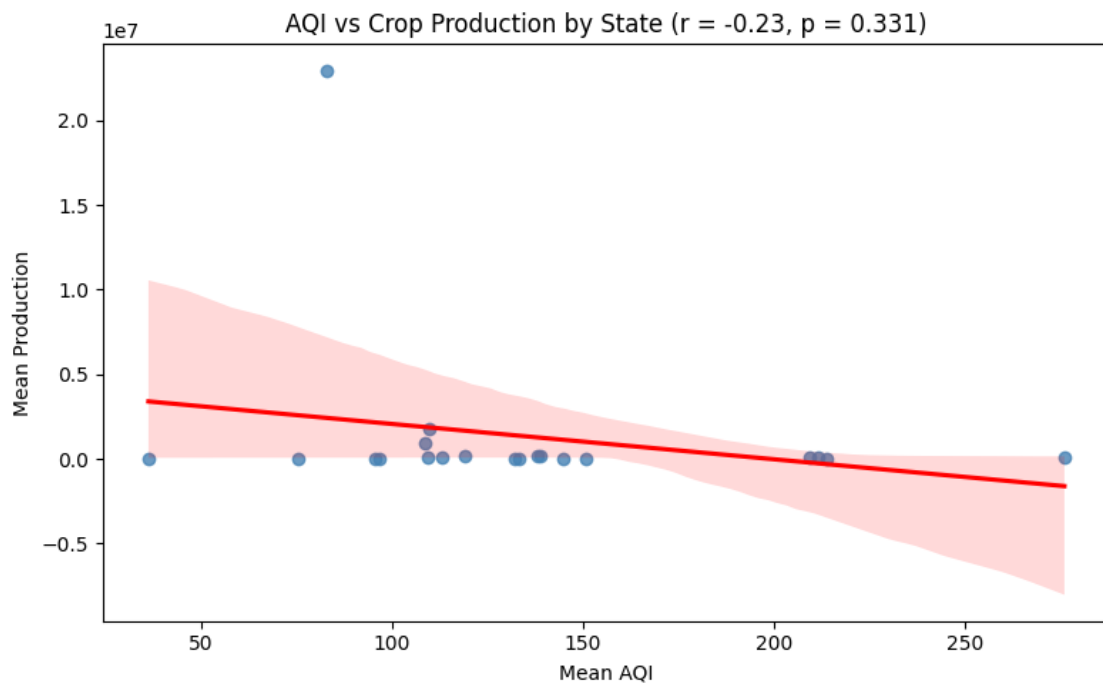
2.5.2 Task B

```
[104]: from scipy import stats

slope, intercept, r, p, se = stats.linregress(merged['AQI'],
merged['Production'])

plt.figure(figsize=(8,5))
sns.regplot(data=merged, x='AQI', y='Production',
            scatter_kws={'color':'steelblue'},
            line_kws={'color':'red'})
plt.title(f'AQI vs Crop Production by State (r = {r:.2f}, p = {p:.3f})')
plt.xlabel('Mean AQI')
plt.ylabel('Mean Production')
plt.tight_layout()
plt.show()

print(f"Correlation (r): {r:.2f}")
print(f"P-value: {p:.3f}")
print(f"Direction: {'Negative' if r < 0 else 'Positive'}")
```



Correlation (r): -0.23

P-value: 0.331

Direction: Negative

$r = -0.23$ indicates a weak negative relationship — as state-level AQI increases, average crop

production tends to decrease slightly. However, this is not strong enough to conclude causation. Factors this dataset cannot account for — rainfall patterns, irrigation access, soil quality, and economic conditions — could explain part or all of this relationship. A funded causal study should control for these confounders before drawing policy conclusions.

2.5.3 Task C

This chart reveals that India’s air quality follows a predictable seasonal cycle — clean during monsoon months, dangerously poor in November at the height of the crop residue burning season. No other chart in this analysis makes the human cause of pollution this visible: the sharp spike from September to November mirrors the harvest calendar almost exactly. For policymakers, this is not a natural disaster — it is a scheduled one, which means it is preventable. What this chart cannot show is whether banning stubble burning alone would be sufficient, since cold winter air trapping pollutants near the ground is a parallel factor this dataset cannot separate from burning activity.